ĐẠI HỌC BÁCH KHOA HÀ NỘI



PROJECT III

Kriky

Trợ lí hỗ trợ giáo viên ra đề và chữa bài

Giảng viên hướng dẫn: Vũ Văn Thiệu

Sinh viên thực hiện: Lê Nguyễn Phước Thành – 20235561

Mục lục

1 Đặt vấn đề **5**
- 1.1 Bối cảnh . . . 5
- 1.2 Vấn đề cần giải quyết . . . 7
- 1.3 Mục tiêu sản phẩm . . . 7
- 1.4 Phạm vi . . . 7

2 Ý tưởng sản phẩm **8**
- 2.1 Luồng tổng thể . . . 8
- 2.2 Hai giai đoạn làm bài của học sinh . . . 10
- 2.3 Làm sao phân tích lỗi của học sinh? . . . 10
- 2.4 Tạo câu hỏi mới dựa trên câu làm sai . . . 10
- 2.5 Thang điểm . . . 10
- 2.6 Giáo viên đưa ra quyết định ở đâu? . . . 11

3 Actor và khái niệm nghiệp vụ **12**
- 3.1 Hai actor . . . 12
  - 3.1.1 Teacher . . . 12
  - 3.1.2 Student . . . 12
- 3.2 Từ điển khái niệm . . . 12
  - 3.2.1 Người dùng, đề và bài làm . . . 12
  - 3.2.2 Thành phần hệ thống . . . 13
  - 3.2.3 Tài liệu . . . 14
  - 3.2.4 Trợ lí . . . 14

4 Các luồng nghiệp vụ **16**
- 4.1 Workflow 1 — Tạo và duyệt đề . . . 18
- 4.2 Workflow 2 — Làm bài (giai đoạn 1) . . . 18
- 4.3 Workflow 3 — Chấm bài . . . 18
- 4.4 Workflow 4 — Chữa bài (giai đoạn 2) . . . 19
  - 4.4.1 Giai đoạn 2 đếm giờ như thế nào? . . . 19
  - 4.4.2 Khi nào kết thúc giai đoạn 2 . . . 19
  - 4.4.3 Giáo viên làm gì? . . . 20
- 4.5 Business Workflows . . . 20

5 Use case **21**
- 5.1 Sáu use case . . . 21
- 5.2 Đặc tả từng use case . . . 23
  - 5.2.1 UC-01 — Tạo đề nháp . . . 23

5.2.2 UC-02 — Review và phát hành đề . . . 23
5.2.3 UC-03 — Làm và nộp bài . . . 23
5.2.4 UC-04 — Chữa bài . . . 23
5.2.5 UC-05 — Tạo câu biến thể . . . 24
5.2.6 UC-06 — Báo cáo Trợ lí giải thích chưa rõ . . . 24

6 Kiến trúc và mô hình dữ liệu 25
6.1 Các service trong hệ thống . . . 25
6.2 Frontend không nhìn thấy bên trong hệ thống . . . 27
6.3 Backend quyết định mọi thứ . . . 27
6.4 Service nào kết nối tới store nào? . . . 27
6.5 Các service nói chuyện với nhau thế nào . . . 28
6.6 Job nào đi qua queue, job nào không . . . 28
6.7 Nếu một service ngừng thì mất gì? . . . 29
6.8 Mô hình dữ liệu . . . 30

7 Chi tiết các service 33
7.1 Backend . . . 33
7.1.1 Vai trò và trách nhiệm . . . 33
7.1.2 Giao job, lấy kết quả và gửi event . . . 33
7.1.3 Xử lý các nghiệp vụ chính . . . 35
7.1.4 Các ràng buộc và cơ chế kiểm soát . . . 38
7.1.5 Xử lý lỗi . . . 39
7.1.6 Các giới hạn hiện tại . . . 40
7.2 Agent . . . 40
7.2.1 Vai trò và trách nhiệm . . . 40
7.2.2 Nhận job, gọi mô hình và trả kết quả . . . 41
7.2.3 Xử lý các task chính . . . 42
7.2.4 Các ràng buộc và cơ chế kiểm soát . . . 44
7.2.5 Xử lý lỗi . . . 45
7.2.6 Các giới hạn hiện tại . . . 46
7.3 Document . . . 46
7.3.1 Vai trò và trách nhiệm . . . 46
7.3.2 Hai worker, ba task và sáu collection . . . 47
7.3.3 Một tệp đi qua service . . . 48
7.3.4 Các ràng buộc và cơ chế kiểm soát . . . 50
7.3.5 Xử lý lỗi . . . 52
7.3.6 Các giới hạn hiện tại . . . 53

Tài liệu tham khảo 54

Danh mục hình vẽ

1.1 Domain Context 6

2.1 Tổng thể một vòng 9

4.1 Activity Overview 17
4.2 Business Workflows 20

5.1 Use Case Diagram 22

6.1 System Architecture 26

7.1 Một lượt hội thoại của giáo viên 36
7.2 Một tệp đi qua `document` 49

Danh mục bảng

2.1 Ba mức điểm của một câu hỏi 11

5.1 Sáu use case và actor chính của từng cái 21

6.1 Năm service của hệ thống 25
6.2 Bảy task gọi Model Provider 29
6.3 Mỗi service ngừng thì mất gì, còn gì 30
6.4 Hai mươi hai bảng dữ liệu 30

7.1 Ba cách `backend` lấy kết quả một job của `agent` 34
7.2 Ràng buộc `backend` giữ 38
7.3 Khi một thao tác của `backend` hỏng 39
7.4 Sáu task của `agent` 41
7.5 Ràng buộc `agent` giữ 44
7.6 Khi một thao tác của `agent` hỏng 45
7.7 Ba task của `document` 47
7.8 Sáu collection của content store 48
7.9 Bảy hàm dựng cây mục lục 50
7.10 Ràng buộc `document` giữ 50
7.11 Khi một thao tác của `document` hỏng 52

Chương 1

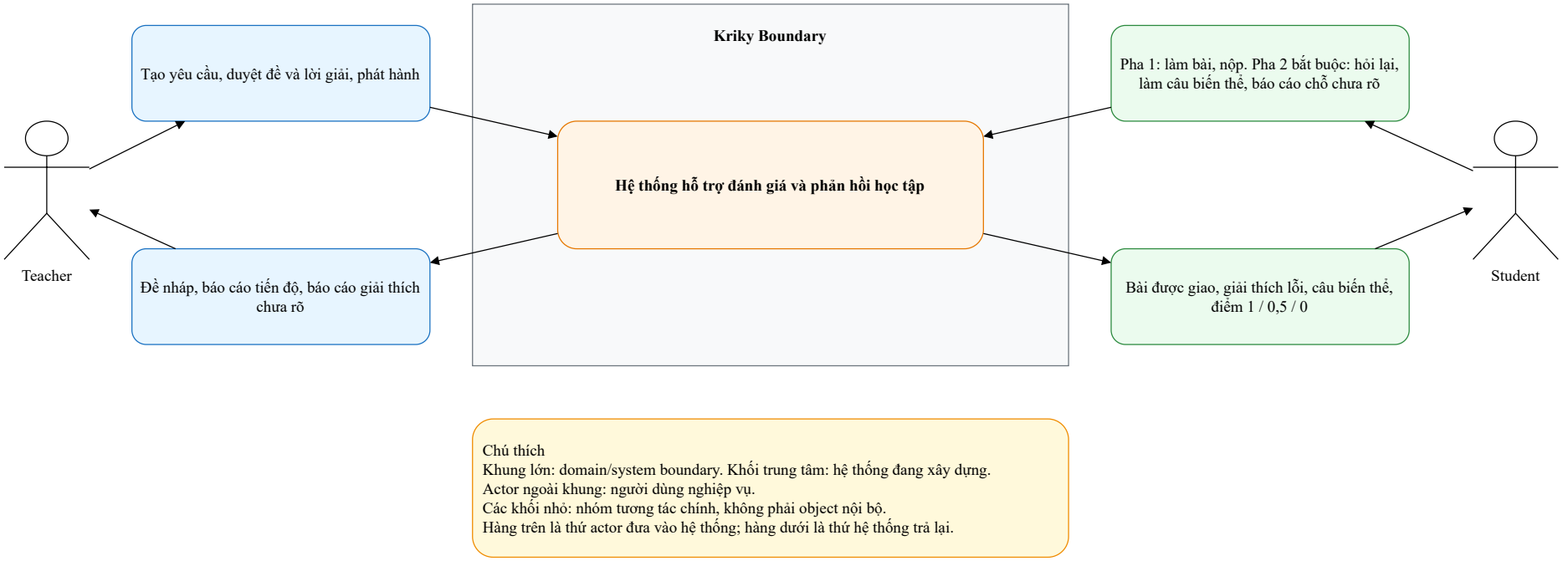
Đặt vấn đề

1.1 Bối cảnh

Ở trường phổ thông, một bài kiểm tra thường kết thúc khi giáo viên trả điểm. Học sinh nhận lại bài, nhìn những câu bị gạch chéo, rồi phần lớn dừng ở đó. Thứ thực sự làm thay đổi kết quả học tập là việc hiểu ra mình sai ở đâu và làm lại được, nhưng việc đó hiếm khi xảy ra, vì nó cần một cuộc trao đổi riêng cho từng em về từng câu.

Giáo viên không phải không muốn làm. Vấn đề nằm ở khối lượng công việc. Một lớp bốn mươi học sinh làm một đề mười câu sẽ sinh ra bốn trăm lượt trả lời, và mỗi lượt sai có thể sai vì một lý do khác nhau. Giải thích riêng cho từng em, rồi ra thêm một câu tương tự để kiểm xem em đã sửa được chưa, là công việc tăng theo sĩ số và không có cách nào rút gọn.

Kriky là hệ thống được xây để gánh phần công việc đó, nhưng không lấy đi quyền quyết định của giáo viên. Hình 1.1 cho thấy ranh giới của hệ thống: hai **actor** (tác nhân, mục 3.2) ở hai phía, cùng những gì đi vào và đi ra.



Hình 1.1: Domain Context — ranh giới hệ thống, hai actor, và những gì đi qua ranh giới ấy

1.2 Vấn đề cần giải quyết

Phần tốn thời gian nhất của giáo viên không phải là gõ ra đề bài, mà là năm việc sau. Cả năm đều là công việc chuyên môn:

- Soạn đề bám đúng mục tiêu học tập, thay vì gom câu hỏi cho đủ số lượng.
- Tạo phương án nhiễu có ý nghĩa. Một phương án sai ngẫu nhiên không nói lên điều gì, còn một phương án nhiễu tốt là phương án mà học sinh mắc một lỗi cụ thể sẽ chọn.
- Chấm bài và viết nhận xét cho từng học sinh.
- Phân định học sinh sai vì thiếu kiến thức, vì suy luận hỏng, vì tính nhầm, hay vì hiểu sai một khái niệm.
- Ra câu biến thể của đúng câu học sinh vừa làm sai, để kiểm xem em đã sửa được chỗ sai đó hay chỉ nhớ được đáp án.

Phía học sinh, nhu cầu ngược lại nhưng khớp vào: phản hồi cần đến nhanh, và bài luyện tập cần đúng với điểm yếu của riêng mình. Một phản hồi đúng nhưng đến sau hai tuần thì không còn tác dụng, vì học sinh đã quên mình nghĩ gì lúc làm bài.

1.3 Mục tiêu sản phẩm

Hệ thống hướng tới ba mục tiêu nghiệp vụ:

1. Hỗ trợ giáo viên tạo đề từ một yêu cầu bằng ngôn ngữ tự nhiên. Một đề có hai nguồn câu hỏi: nội dung sinh mới, và **Question Bank** (ngân hàng câu hỏi, mục 3.2).
2. Phân tích lỗi sai hoặc hiểu nhầm khái niệm của học sinh thông qua cuộc trò chuyện thay vì một câu nhận xét.
3. Tạo các câu hỏi mới từ các câu mà học sinh làm sai ngay trong cùng bài kiểm tra.

1.4 Phạm vi

Hệ thống dựng trọn một vòng nghiệp vụ, từ đầu đến cuối:

- Giáo viên tạo, xem lại và phát hành đề. Việc duyệt bao gồm cả lời giải và ánh xạ từ phương án nhiễu sang lỗi, không chỉ câu hỏi và đáp án.
- Giai đoạn 1: Học sinh làm bài, nộp bài, và hệ thống chấm bài để biết học sinh đang thiếu kiến thức ở phần nào.
- Giai đoạn 2: Học sinh trò chuyện với trợ lí Kriky để hiểu cách làm bài và tại sao lại sai. Sau đó hệ thống tạo câu hỏi mới, học sinh làm lại cho tới khi hiểu được cách làm của dạng bài.

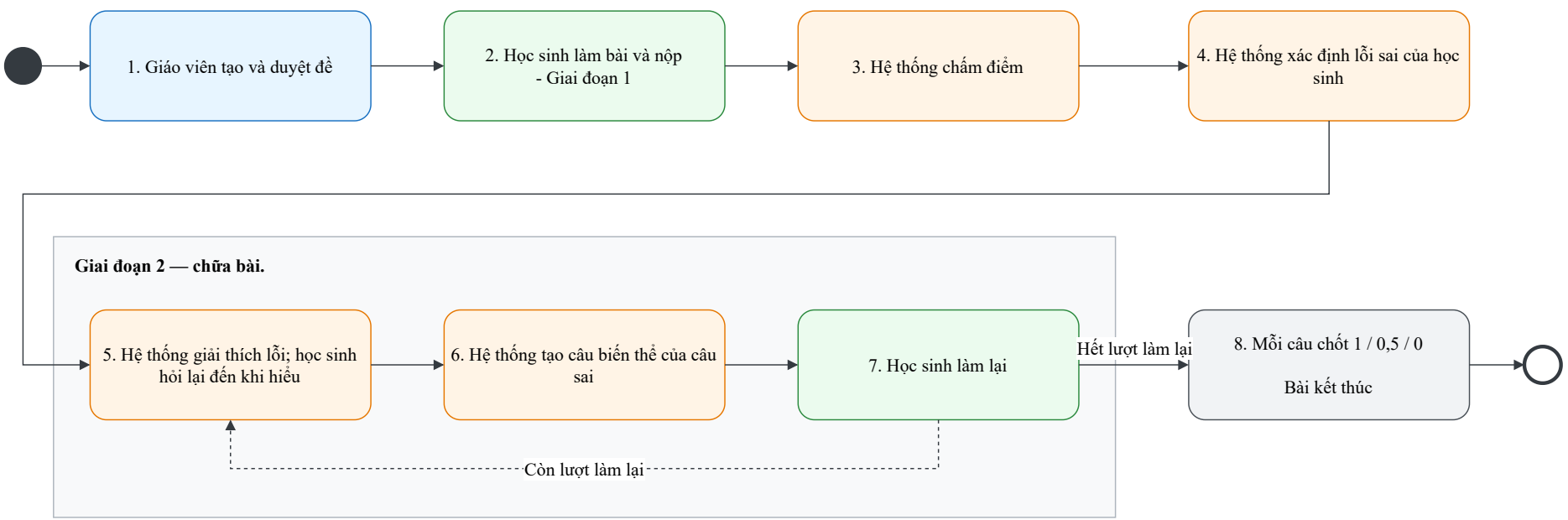
Chương 2

Ý tưởng sản phẩm

2.1 Luồng tổng thể

Ý tưởng lõi không nằm ở việc sinh câu hỏi, mà ở việc khép kín một vòng tám bước:

1. Giáo viên tạo và duyệt đề.
2. Học sinh làm bài.
3. Hệ thống chấm điểm.
4. Hệ thống xác định lỗi sai của học sinh.
5. Hệ thống giải thích lỗi, học sinh hỏi lại đến khi hiểu.
6. Hệ thống tạo câu biến thể của câu sai.
7. Học sinh làm lại.
8. Mỗi câu chốt ở một trong ba mức điểm, và bài kết thúc.



Hình 2.1: Tổng thể một vòng

2.2 Hai giai đoạn làm bài của học sinh

Bước 2, 7 và 8 của hình 2.1.

Một bài kiểm tra có hai giai đoạn: giai đoạn 1 là làm bài và nộp, giai đoạn 2 là chữa những câu đã sai. Nộp bài kết thúc giai đoạn 1 chứ không kết thúc bài. Bài chỉ kết thúc khi giai đoạn 2 xong, hoặc khi hạn của giai đoạn 2 hết.

Kéo theo đó là một hệ quả cần nói rõ: một bài kiểm tra có hai trạng thái hoàn thành khác nhau, *đã nộp* và *đã hoàn thành*, và chúng không thay thế cho nhau được.

2.3 Làm sao phân tích lỗi của học sinh?

Bước 4 của hình 2.1.

Với hình thức trắc nghiệm, việc chấm chỉ là một phép so. Khó khăn nằm ở chẩn đoán lỗi sai của học sinh, bởi cùng chọn một phương án sai có thể do nhiều lỗi khác nhau sinh ra. Nếu hệ thống đoán lỗi rồi đi giải thích một lỗi học sinh không hề mắc thì nó còn tệ hơn im lặng: em vừa mất thời gian, vừa được dạy sai chỗ.

Vì vậy, mỗi phương án sai phải mang theo lỗi mà nó đại diện, và được soạn cùng câu hỏi chứ không suy ra lúc chấm.

Hai luật đi kèm:

- Mỗi câu hỏi phải mang theo lời giải có nhiều hơn một cách làm.
- Số phương án không cố định. Một câu có thể có ba, bốn hay năm lựa chọn tuỳ từng câu, nhưng chỉ đúng một lựa chọn là đáp án đúng. Mọi phương án còn lại đều là nhiễu, nên đều phải có lỗi gắn kèm.

2.4 Tạo câu hỏi mới dựa trên câu làm sai

Bước 6 và 7 của hình 2.1

Khi học sinh sai câu 3, hệ thống sinh câu 3̀, gọi là câu biến thể. Thứ cần chữa là một lỗi cụ thể vừa xảy ra, không phải một chủ đề.

Câu biến thể giữ nguyên dạng đề và cách làm, chỉ đổi dữ kiện, con số và cách hỏi. Đó là cách duy nhất kiểm được rằng học sinh đã sửa đúng chỗ mình sai, chứ không phải chỉ nhớ được đáp án.

2.5 Thang điểm

Bước 8 của hình 2.1.

Ba mức thay vì hai, vì hai mức sẽ biến việc chữa bài thành công việc không được có ý nghĩa. Một học sinh sửa được lỗi của mình đã học đúng thứ mà bài kiểm tra muốn dạy, và mức giữa là chỗ duy nhất ghi nhận điều đó. Vì vậy giai đoạn 2 chỉ cộng chứ không bao giờ trừ: điểm tính ngay sau khi nộp là sàn.

Bảng 2.1: Ba mức điểm của một câu hỏi

Điểm	Nghĩa
1	Đúng ngay ở giai đoạn 1
0,5	Sai ở giai đoạn 1 nhưng chữa được ở giai đoạn 2
0	Sai và không chữa được

2.6 Giáo viên đưa ra quyết định ở đâu?

Bước 1 của hình 2.1.

Với nội dung đề, trợ lí viết còn giáo viên quyết nội dung đó có đến tay học sinh hay không. Giáo viên bắt buộc tham gia ở hai điểm, một ở đầu ra và một ở đầu vào:

1. Trợ lí hỏi lại khi chưa đủ thông tin, thay vì tự đoán (đầu vào).
2. Duyệt đề trước khi phát hành (đầu ra).

Chương 3

Actor và khái niệm nghiệp vụ

3.1 Hai actor

Hệ thống có đúng hai **actor** (tác nhân) nghiệp vụ: **Teacher** (giáo viên) và **Student** (học sinh).

3.1.1 Teacher

Teacher là người đưa ra các quyết định. Teacher có thể:

- Nhập yêu cầu tạo đề.
- Xem và chỉnh sửa đề do trợ lí tạo.
- Duyệt đề trước khi phát hành.
- Phát hành đề cho một hoặc nhiều lớp.
- Xem kết quả làm bài của học sinh.
- Xem báo cáo học sinh gửi về chỗ trợ lí giải thích chưa rõ.

3.1.2 Student

Student là người nhận đề, làm bài và luyện tập. Student có thể:

- Xem bài kiểm tra được giao.
- Chọn đáp án và nộp bài (giai đoạn 1).
- Nghe giải thích lỗi và hỏi lại đến khi hiểu (giai đoạn 2).
- Làm câu biến thể của những câu đã sai (giai đoạn 2).
- Báo cáo chỗ trợ lí giải thích chưa rõ.

3.2 Từ điển khái niệm

3.2.1 Người dùng, đề và bài làm

- **Actor**: Hệ thống có hai actor: *Teacher* là giáo viên, *Student* là học sinh. Báo cáo chỉ dùng hai chữ tiếng Anh này khi nói về actor và trong sơ đồ; khi kể thì gọi là giáo viên, học sinh.
- **Use case**: Một chức năng hệ thống mang lại cho actor.
- **Workflow**: Luồng nghiệp vụ, tức chuỗi hoạt động mà các actor và hệ thống lần lượt làm để đạt một mục tiêu, như tạo và duyệt một đề. Một workflow có thể gồm nhiều use case.

- **Assessment**: Bài kiểm tra, hay bộ câu hỏi được giao cho học sinh.
- **Question**: Câu hỏi trong một assessment.
- **Distractor**: Phương án sai *có chủ đích*, đại diện cho một lỗi hoặc một hiểu nhầm cụ thể. Không phải đáp án nhiễu ngẫu nhiên.
- **Lời giải**: Cách làm đi kèm mỗi câu hỏi, bắt buộc có nhiều hơn một cách. Trợ lí dựa vào lời giải để giải thích cho học sinh.
- **Draft**: Đề đang soạn, chưa duyệt. Câu hỏi ở trạng thái này sửa và xoá được tự do.
- **Approve**: Chốt rằng đề đã dùng được. Đây là một *trạng thái*, không phải một thao tác: nó có cả chiều ngược, gọi là *unapprove*, và chỉ đề đã approve mới phát hành được.
- **Question Bank**: Ngân hàng câu hỏi riêng của từng giáo viên.
- **Class**: Lớp học có sẵn danh sách học sinh. Giáo viên tạo lớp. Phát hành đề phải chọn một lớp đã tạo từ trước.
- **Publish** và **Publication**: *Publish* là phát hành một đề cho một lớp. Một *Publication* là bản ghi sinh ra từ việc đó.
- **Giai đoạn 1**: Phần làm bài và nộp.
- **Giai đoạn 2**: Phần chữa bài.
- **opens_at** / **closes_at** / **remediation_deadline**: Ba mốc thời gian của một *Publication*. Hai mốc đầu là thời điểm mở và đóng giai đoạn 1; mốc thứ ba là thời điểm đóng giai đoạn 2.
- **Submission**: Việc nộp bài, và là mốc kết thúc *giai đoạn 1* — không phải mốc kết thúc bài kiểm tra.
- **Attempt**: Một lần làm bài của một học sinh trên một đề, trải qua **cả hai giai đoạn**. Mỗi học sinh có duy nhất một Attempt cho mỗi đề — em chưa vào làm thì không có Attempt nào. Nó xong khi giai đoạn 2 hoàn thành.
- **Mark** và **Score**: *Mark* là điểm của *một câu*; *score* là điểm của *cả bài*.
- **Misconception**: Hiểu nhầm khái niệm, hoặc lỗi sai có tính lặp lại.
- **Feedback**: Phản hồi / nhận xét của trợ lí cho những thắc mắc của học sinh.
- **Lượt**: Một lượt cho phép học sinh làm lại các câu sai của giai đoạn trước / lượt trước. Khác với một *lượt hội thoại*: một tin nhắn cùng mọi thứ trợ lí làm để đáp lại tin ấy.
- **Câu biến thể**: Câu sinh từ *câu học sinh làm sai ở giai đoạn 1*: giữ cấu trúc và lỗi cần kiểm, chỉ đổi dữ kiện.

3.2.2 Thành phần hệ thống

- **Service**: Một đơn vị chạy độc lập trong hệ thống. Hiện tại có năm service: `frontend`, `backend`, `agent`, `document`, `ingest`.
- **API** (Application Programming Interface): Giao diện lập trình, tức cách một chương trình gọi một chương trình khác. Giữa các service, chỉ `frontend` gọi `backend` theo cách này; các service nội bộ không gọi API để giao tiếp với nhau.
- **Gateway**: `backend` là gateway của cả hệ thống: không màn hình nào của `frontend` gọi thẳng `agent`, `document` hay `ingest`.
- **VPC** và **Subnet**: *VPC* là đám mây riêng ảo, tức một vùng mạng riêng tách khỏi bên ngoài; nó chứa mọi service trừ `frontend`. Nó chia thành các *subnet*, tức các vùng mạng con: chỉ những gì nằm ở *public subnet* (vùng mạng công khai) mới nhận được kết nối từ bên ngoài, còn những gì nằm ở *private subnet* (vùng mạng nội bộ) thì không. Hình kiến trúc ở chương 6 đặt `backend` ở public subnet, mọi service và kho khác ở private subnet.
- **Job** và **Task**: Hai thứ khác nhau. Một *task* là *tên* của một loại việc. Một *job* là một lần chạy của task ấy, mỗi job có một mã riêng.
- **Queue**: Nơi một service đẩy job vào và một service khác lấy ra làm. Job nằm lại trên queue

đến khi có ai lấy, nên tắt máy giữa chừng không mất việc.
- **Worker**: Tiến trình ngồi nghe một queue, lấy job xuống và chạy. Nó không nhận lời gọi từ ngoài; thứ duy nhất nó đọc là queue. Một service có thể có nhiều worker: `document` có một worker đọc chữ của tệp và một worker dựng cây mục lục.
- **Byte**: Đơn vị cơ bản để đo và lưu dữ liệu trong máy; mọi tệp, dù là PDF hay ảnh chụp, đều là một dãy byte. Báo cáo nói *byte của tệp*, tức dữ liệu gốc, để chỉ chính tệp giáo viên tải lên, nguyên như lúc tải, khác với chữ hệ thống đọc ra từ nó.
- **Store**: Kho dữ liệu mà service đọc và ghi. Hệ thống có bốn store thuộc bốn loại khác nhau, nên mỗi loại mang chữ riêng: *database* giữ dữ liệu có cấu trúc, *object store* giữ byte của tệp, *content store* giữ nội dung đã xử lí của tài liệu, như chữ từng trang và cây mục lục, và *queue* giữ job đang chờ.
- **Collection**: Nhóm bản ghi cùng loại trong content store, giống một bảng của database.
- **Credential**: Khoá cho phép một service vào một store để đọc / ghi.
- **Pub/sub** và **Channel**: *Pub/sub* là cách một service phát tin lên một *channel*, tức một kênh có tên: ai đang nghe channel ấy thì nhận, ai không nghe thì mất. Khác queue ở chỗ nó **không giữ lịch sử**, nên hệ thống chỉ phát qua nó những thứ mất được.
- **SSE** và **Polling**: Hai cách để màn hình cập nhật thời gian thực (realtime). Với *SSE*, `backend` giữ một kết nối mở và gửi tin xuống qua đó mỗi khi có gì đổi. Với *polling*, màn hình tự hỏi lại theo một chu kì. Hệ thống hiện tại dùng *SSE*.
- **Event**: Một tin `backend` gửi xuống màn hình qua SSE.

3.2.3 Tài liệu

Nhóm này đi theo một tài liệu, từ tệp giáo viên tải lên tới những mẩu chữ trợ lí đọc:

- **Document**: Tài liệu giáo viên tải lên, là một tệp PDF hoặc tệp chữ thuần `.txt`, `.md`. Nó nằm trong kho lưu trữ riêng của từng tài khoản, cung cấp kiến thức và giới hạn phạm vi ra đề. Một tài liệu chỉ dùng được khi hệ thống đã đọc ra chữ của nó, kể cả đọc từ ảnh nếu tệp là bản chụp, và đã dựng xong *cây mục lục*.
- **OCR** (Optical Character Recognition): Đọc chữ từ ảnh trong tài liệu.
- **Bản thô**: Những gì `document` đọc được từ một tệp, chia theo trang và theo khối chữ, trước khi có cây mục lục. Bản thô được ghi vào content store trước, rồi cây mục lục mới được dựng từ nó.
- **Cây mục lục**: Mục lục của một *Document* mà hệ thống tự dựng từ nội dung của tệp. Cây có tối đa ba cấp, ví dụ: chương, bài và mục.
- **Chunk**: Một mẩu chữ ngắn cắt từ bản thô, thường thuộc về đúng một nút của cây mục lục. Khi trợ lí cần đọc một bài của tài liệu, `backend` lấy các chunk của bài ấy và các mục bên dưới đưa cho nó, chứ không đưa cả tài liệu.
- **Scope**: Phạm vi ra đề trong một *Document*: các nút giáo viên chọn trên cây mục lục, như một chương hay vài bài, cùng mọi mục nằm dưới chúng. Scope trỏ vào nút, không trỏ vào số trang.

3.2.4 Trợ lí

- **Model Provider**: Nhà cung cấp mô hình ngôn ngữ, nằm ngoài hệ thống. Chỉ hai service gọi tới nó: `agent`, để soạn câu và trò chuyện, và `document`, khi dựng cây mục lục.
- **Tool**: Một việc có tên mà `agent` được phép *đề xuất* chạy, kèm tham số. Có tool tra cứu, như xem lớp, xem kết quả của một đề, đọc mục lục và chữ của tài liệu; và tool ghi, như mở đề, soạn câu, thay một câu. Danh sách tool là một **ràng buộc**, không phải một tiện ích:

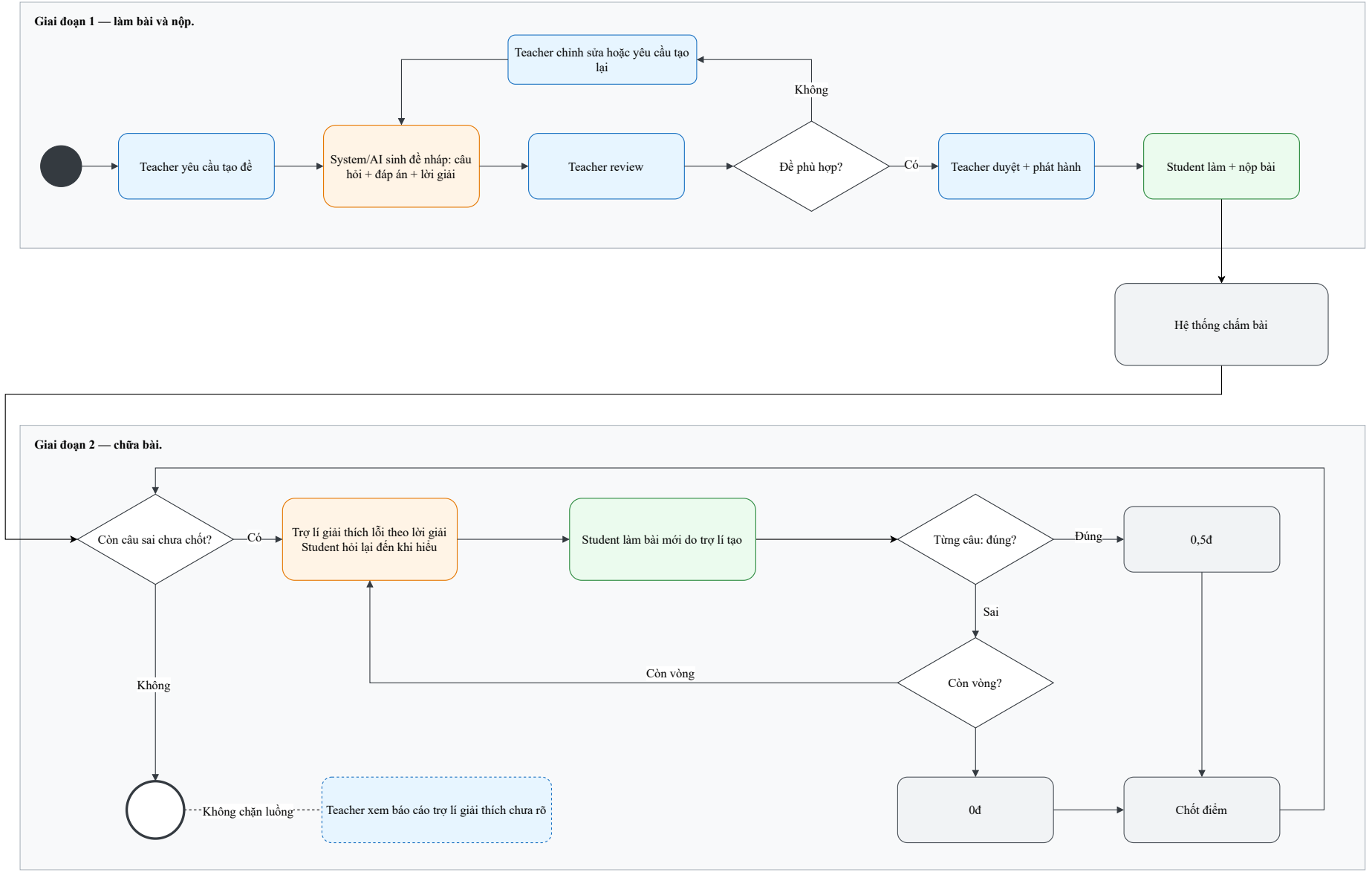
duyệt đề và phát hành đề cố ý không có tool nào, nên `agent` không có đường nào để đề xuất chúng.

- **Dàn ý**: Kế hoạch của một đề mà trợ lí đưa ra *trước* khi soạn câu nào. Mỗi mục của dàn ý nói mục ấy có mấy câu, kiểm kiến thức gì, dùng công thức nào, làm bài theo cách nào, và dựa vào bài nào của tài liệu nếu có. Giáo viên đồng ý thì câu hỏi mới được soạn.
- **Plan**: Danh sách việc trợ lí định làm cho một tin nhắn của giáo viên, mỗi việc là một *tool* kèm tham số.
- **Graph**: Cách `agent` và `document` tổ chức một job có gọi mô hình: một chuỗi các hàm cố định, dựng bằng thư viện LangGraph. Mỗi hàm làm một phần rồi trao kết quả cho hàm sau; chuỗi có thể quay lại một hàm trước, như khi hàm viết câu hỏi không qua được hàm kiểm tra và phải viết lại.

Chương 4

Các luồng nghiệp vụ

Nghiệp vụ của hệ thống gồm bốn luồng. Hình 4.1 và hình 2.1 mô tả cùng một luồng, nhưng ở hai mức chi tiết khác nhau.



Hình 4.1: Activity Overview — toàn bộ vòng, hai giai đoạn nằm trong hai khung riêng

4.1 Workflow 1 — Tạo và duyệt đề

Mục tiêu. Giúp giáo viên tạo được đề phù hợp với mục tiêu học tập, nhưng giáo viên vẫn là người duyệt cuối cùng trước khi phát hành.

Luồng chính.

1. **Teacher** (giáo viên) nhập yêu cầu tạo đề.
2. Teacher cung cấp ràng buộc: chủ đề, độ khó, số lượng câu hỏi, loại bài kiểm tra, thời gian làm bài.
3. `System/AI` phân tích yêu cầu.
4. `System/AI` tạo đề nháp gồm câu hỏi, phương án trả lời, đáp án đúng, lời giải và **metadata** (thông tin mô tả) cơ bản.
5. Teacher xem đề nháp.
6. Teacher chỉnh sửa thủ công, hoặc yêu cầu tạo lại một phần.
7. Teacher duyệt đề.
8. Teacher phát hành đề cho **Student** (học sinh).

Điểm kiểm soát. Teacher phải duyệt đề trước khi đề được phát hành. Đề kiểm tra ảnh hưởng trực tiếp đến việc đánh giá học sinh, nên trợ lí không được tự phát hành.

4.2 Workflow 2 — Làm bài (giai đoạn 1)

Mục tiêu. Cho phép học sinh làm bài và ghi nhận bài nộp để hệ thống đánh giá.

Luồng chính.

1. Student nhận **assessment** (bài kiểm tra) đã được Teacher phát hành.
2. Student vào làm, *tới hết* giờ đóng.
3. Student trả lời câu hỏi trắc nghiệm.
4. Student nộp bài.
5. Hệ thống ghi nhận **submission** (việc nộp bài) để đánh giá.

Ba tham số thời gian. Giai đoạn 1 có ba tham số thời gian độc lập: thời gian làm bài (`phase1_minutes`), giờ mở (`opens_at`) và giờ đóng (`closes_at`).

Luật là: giờ đóng chặn việc *vào làm*, không chặn việc nộp. Học sinh vào tới hết giờ đóng vẫn được. Ai đã vào rồi thì không bị dừng giữa chừng, và được đủ thời gian làm bài kể cả khi tràn qua giờ đóng. Ví dụ, đóng lúc 18:00 với thời gian làm bài 15 phút thì bài cuối có thể nộp lúc 18:15.

4.3 Workflow 3 — Chấm bài

Mục tiêu. Biết được những câu mà học sinh làm sai, từ đó trả lời các thắc mắc của học sinh thông qua cuộc trò chuyện và tạo ra các câu hỏi biến thể để kiểm tra độ hiểu bài.

Luồng chính.

1. Học sinh nộp bài.
2. Hệ thống **so** phương án đã chọn với đáp án đúng của câu hỏi đã duyệt. Đây là một phép so, không phải một phán đoán — nên không có gì để mà không chắc.
3. Điểm giai đoạn 1 chốt ngay và học sinh thấy ngay.

Hai điều kiện phải có sẵn từ lúc duyệt đề.

- Mỗi phương án nhiễu phải đại diện cho một lỗi cụ thể, soạn cùng lúc với câu hỏi. Phương án nhiễu mà không đại diện cho một lỗi cụ thể thì không giúp học sinh tiến bộ.
- Mỗi câu hỏi phải kèm lời giải. Việc chấm không dùng tới nó, nhưng phương án nhiễu phải được tạo dựa trên lời giải thì mới có ý nghĩa và trợ lí phải dựa vào lời giải để giải thích cho học sinh chứ không tự nghĩ ra cách giải mới. Chính vì thế lời giải phải tồn tại từ lúc duyệt đề chứ không sinh ra lúc học sinh hỏi.

4.4 Workflow 4 — Chữa bài (giai đoạn 2)

Mục tiêu. Bài kiểm tra chỉ kết thúc khi học sinh hiểu bài và sửa được lỗi họ mắc phải.

Luồng chính.

1. Học sinh trò chuyện với trợ lí Kriky về lỗi sai, chỗ nào chưa hiểu, cách giải như thế nào.
2. Kriky dựa vào *lời giải đã soạn kèm câu hỏi*. Học sinh hỏi lại đến khi hiểu. Phần này **không tính giờ**.
3. Trong lúc đó, hệ thống sinh câu biến thể của câu mà học sinh làm sai.
4. Học sinh bấm nút làm bài mới. Đồng hồ của lượt bắt đầu chạy.
5. Học sinh làm các câu mới trong lượt.
6. Câu nào đúng thì chốt **0,5** điểm; câu nào còn sai thì sang vòng tiếp.
7. Mỗi câu tối đa **ba vòng**. Hết ba vòng mà vẫn sai thì chốt **0** điểm.
8. Bài kết thúc khi mọi câu đã chốt, hoặc khi hết hạn giai đoạn 2.

4.4.1 Giai đoạn 2 đếm giờ như thế nào?

Giai đoạn 2 nhận hai tham số: số phút mỗi câu (`phase2_minutes_per_question`), và hạn kết thúc giai đoạn 2 (`remediation_deadline`).

Thời lượng của một lượt bằng số phút mỗi câu nhân với số câu còn sai trong lượt đó, nên sai hai câu với thời lượng làm lại 5 phút mỗi câu thì lượt ấy dài 10 phút.

Phần giải thích không tính giờ. Đồng hồ chỉ bắt đầu khi học sinh tự bấm nút làm bài mới.

4.4.2 Khi nào kết thúc giai đoạn 2

Hạn kết thúc giai đoạn 2 là một mốc tuyệt đối do giáo viên đặt, chung cho cả lớp, và dài hơn hẳn hạn của giai đoạn 1, chẳng hạn là cuối ngày.

Hết hạn thì dừng, kể cả khi học sinh đang làm dở một lượt, và câu chưa xong nhận 0 điểm.

Đây là chỗ giai đoạn 2 làm ngược với giai đoạn 1. Ở giai đoạn 1, học sinh đã vào thì không bị dừng giữa chừng; ở giai đoạn 2, hết hạn là dừng. Hai luật nghịch nhau một cách có chủ ý:

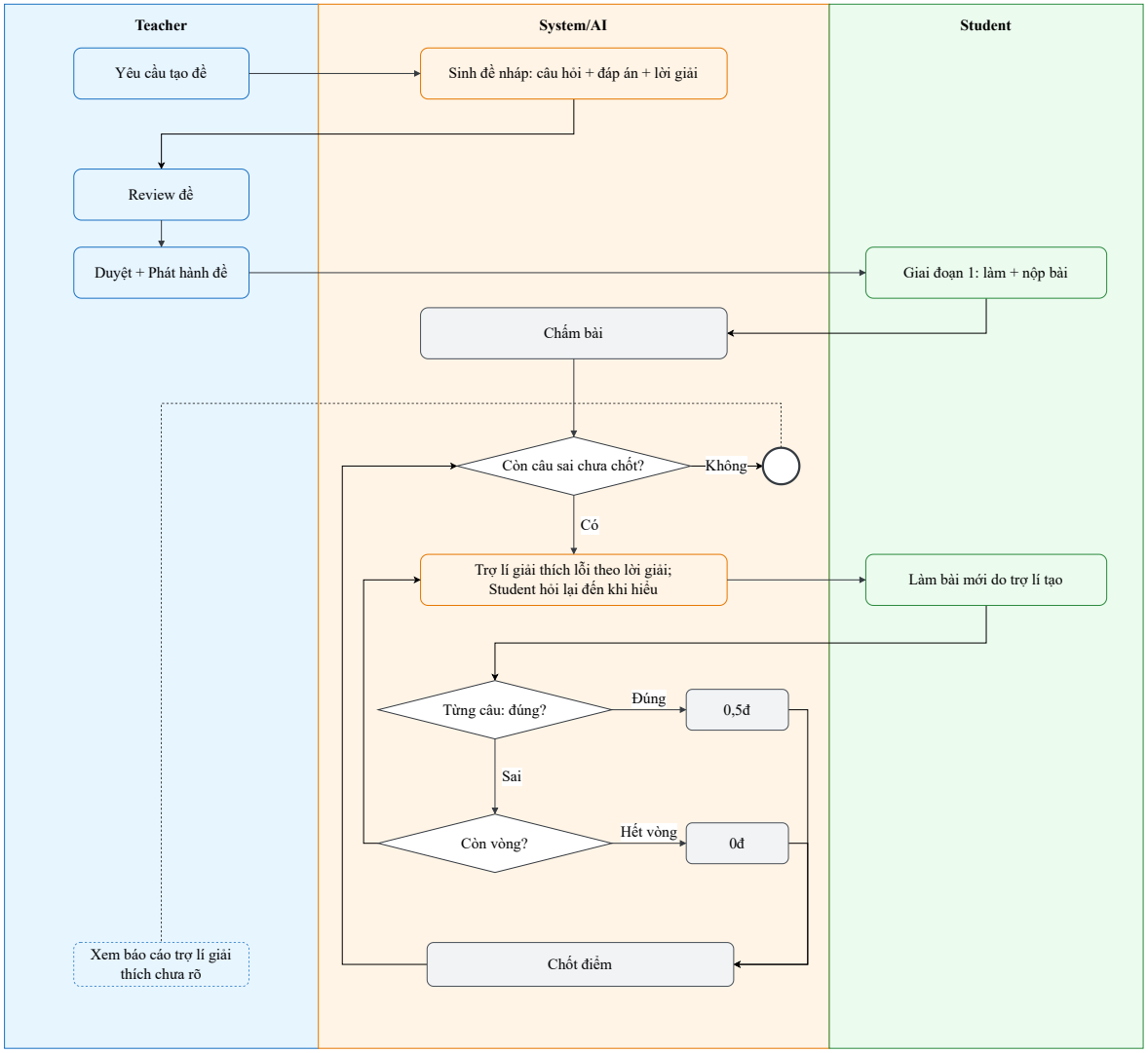
thời gian ở giai đoạn 1 là một lượng hệ thống cấp cho từng người kể từ lúc em vào, còn hạn của giai đoạn 2 là một mốc chung cả lớp nhìn thấy suốt ngày — không cắt thì mốc ấy không tồn tại.

4.4.3 Giáo viên làm gì?

Học sinh báo cáo một câu hoặc một đoạn hội thoại là *giải thích chưa rõ*. Giáo viên xem được các báo cáo này và giải thích để đưa vào bài dạy trên lớp.

4.5 Business Workflows

Hình 4.1 cho thấy luồng tổng quan; hình 4.2 cho thấy trách nhiệm của từng vị trí.



Hình 4.2: Business Workflows

Chương 5

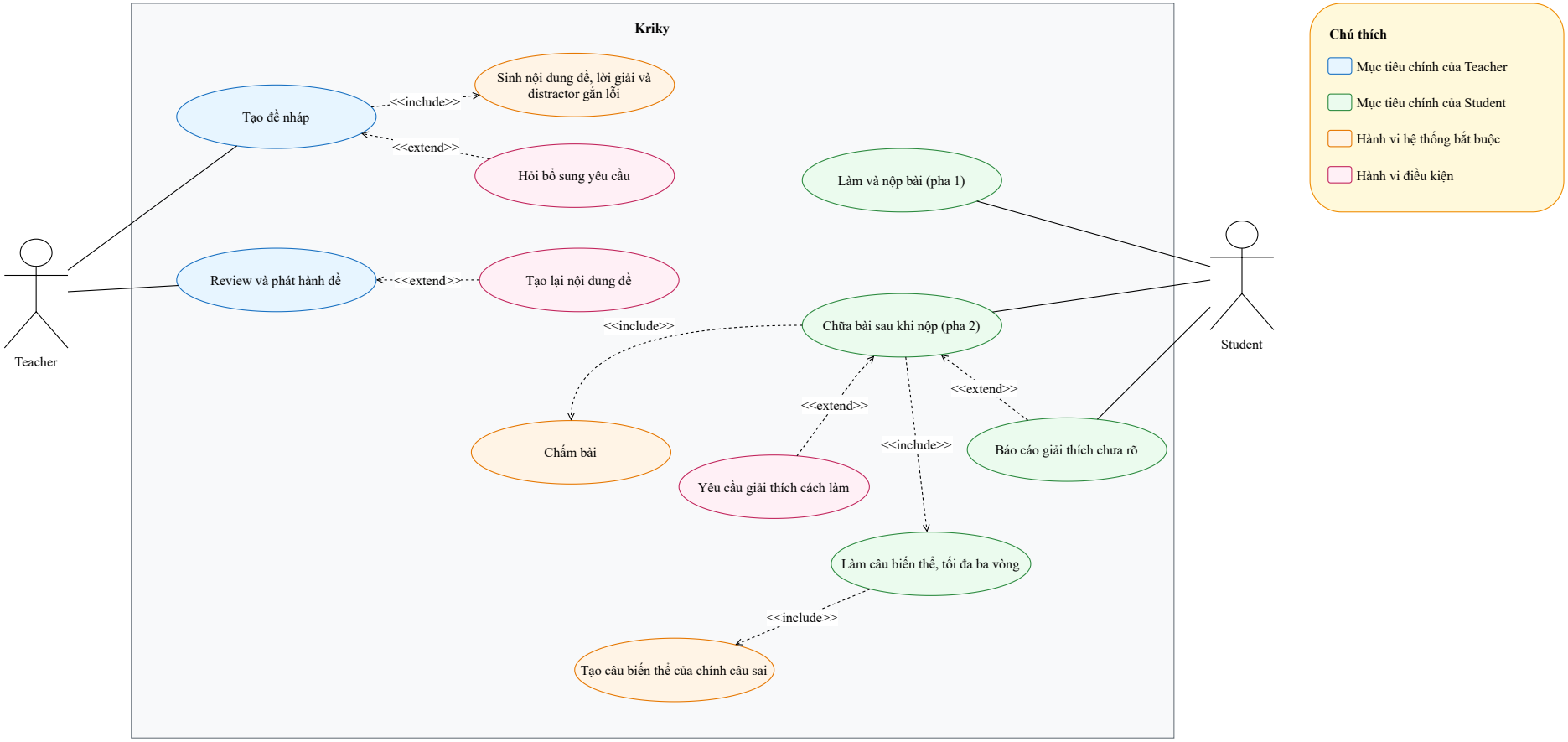
Use case

5.1 Sáu use case

Bảng 5.1: Sáu use case và actor chính của từng cái

Mã	Actor chính	Tên
UC-01	Teacher	Tạo đề nháp
UC-02	Teacher	Review và phát hành đề
UC-03	Student	Làm và nộp bài (giai đoạn 1)
UC-04	Student	Chữa bài sau khi nộp (giai đoạn 2)
UC-05	—	Hệ thống tạo câu biến thể
UC-06	Student	Báo cáo chỗ trợ lí giải thích chưa rõ

UC-05 không có **actor** (tác nhân) trực tiếp: đó là hành vi hệ thống, được `<<include>>` từ UC-04.



Hình 5.1: Use Case Diagram

5.2 Đặc tả từng use case

5.2.1 UC-01 — Tạo đề nháp

Điều kiện trước. **Teacher** (giáo viên) biết chủ đề hoặc mục tiêu học tập cần kiểm tra, và mô tả được yêu cầu ở mức đủ hiểu.

Luồng chính. Teacher nhập yêu cầu và các ràng buộc (chủ đề, số câu, độ khó, thời gian làm bài, loại bài). Hệ thống phân tích, sinh đề nháp gồm câu hỏi, các phương án, đáp án đúng, và lời giải rồi hiển thị cho Teacher.

Quan hệ. *Tạo đề nháp* `<<include>>` *Sinh nội dung đề*. *Hỏi bổ sung yêu cầu* `<<extend>>` *Tạo đề nháp*, khi yêu cầu ban đầu chưa đủ rõ — đây chính là một trong hai nơi Giáo viên ra quyết định ở mục 2.6.

5.2.2 UC-02 — Review và phát hành đề

Điều kiện trước. Đã có đề nháp.

Luồng chính. Teacher xem từng câu hỏi, xem đáp án đúng, lời giải và phương án sai; chỉnh sửa nếu cần; duyệt đề; rồi phát hành với sáu tham số. Mỗi lần phát hành cho một lớp sinh ra một **Publication** (bản ghi phát hành).

Quan hệ. *Tạo lại nội dung đề* `<<extend>>` *Review và phát hành đề*, khi Teacher thấy một phần hoặc toàn bộ đề chưa phù hợp.

5.2.3 UC-03 — Làm và nộp bài

Điều kiện trước. Bài đã được phát hành và **Student** (học sinh) có quyền truy cập.

Luồng chính. Student mở bài, đọc câu hỏi, chọn đáp án, nộp bài. Hệ thống ghi nhận, và chấm bài.

Sơ đồ *không* dùng `<<include>>` từ *Làm và nộp bài* sang *Chấm bài*: chấm bài là bước xử lý *sau* khi có bài nộp, không phải hành vi bắt buộc để Student hoàn thành mục tiêu nộp bài của mình.

Kết quả. Bài nộp được ghi nhận và được chấm. **Nộp bài không kết thúc bài kiểm tra** — nó kết thúc giai đoạn 1.

5.2.4 UC-04 — Chữa bài

Điều kiện trước. Student đã nộp bài ở giai đoạn 1 và hệ thống đã chấm; chưa quá hạn kết thúc giai đoạn 2.

Luồng chính. Student mở kết quả; mở màn hỏi trợ lí, nơi mọi câu sai nằm cùng một màn và phần giải thích không tính giờ; bấm nút làm bài mới để khởi động đồng hồ của lượt; làm câu biến thể của từng câu sai. Câu đúng thì chốt 0,5; câu còn sai thì sang vòng tiếp, tối đa ba vòng, hết ba vòng thì chốt 0.

Ngoại lệ. Hết hạn giai đoạn 2 giữa một lượt thì lượt bị *dừng* và các câu còn dở nhận 0 điểm. Student không mở giai đoạn 2 lần nào thì mọi câu sai nhận 0 điểm.

Quan hệ. *Chữa bài* `<<include>>` *Chấm bài và tra cứu lỗi sai*, và `<<include>>` *Làm câu biến thể, tối đa ba vòng*. *Yêu cầu giải thích cách làm* và *Báo cáo giải thích chưa rõ* đều `<<extend>>` từ nó.

Quan hệ với *Làm câu biến thể* là `<<include>>` chứ không phải `<<extend>>`, **vì giai đoạn 2 bắt buộc**: nó không phải một nhánh có thể không xảy ra. Đây là ví dụ rõ nhất trong cả sơ đồ về việc một quyết định sư phạm đổi luôn kiểu quan hệ trên hình.

5.2.5 UC-05 — Tạo câu biến thể

Điều kiện trước. Một câu ở giai đoạn 1 bị làm sai.

Luồng chính. Hệ thống lấy câu Student làm sai, sinh biến thể của *chính câu đó* — giữ nguyên cấu trúc và lỗi cần kiểm, thay dữ kiện — rồi đưa thẳng tới Student. Kết quả của câu biến thể quyết định câu gốc chốt 0,5 hay sang vòng tiếp.

5.2.6 UC-06 — Báo cáo Trợ lí giải thích chưa rõ

Điều kiện trước. Student đang ở giai đoạn 2, và trợ lí đã trả lời ít nhất một lần.

Luồng chính. Student bấm nút báo cáo “Trợ lí giải thích chưa rõ”. Hệ thống ghi lại **cả đoạn hội thoại của bài kiểm tra đó** kèm những câu Student làm sai để giáo viên có thể xem được.

Chương 6

Kiến trúc và mô hình dữ liệu

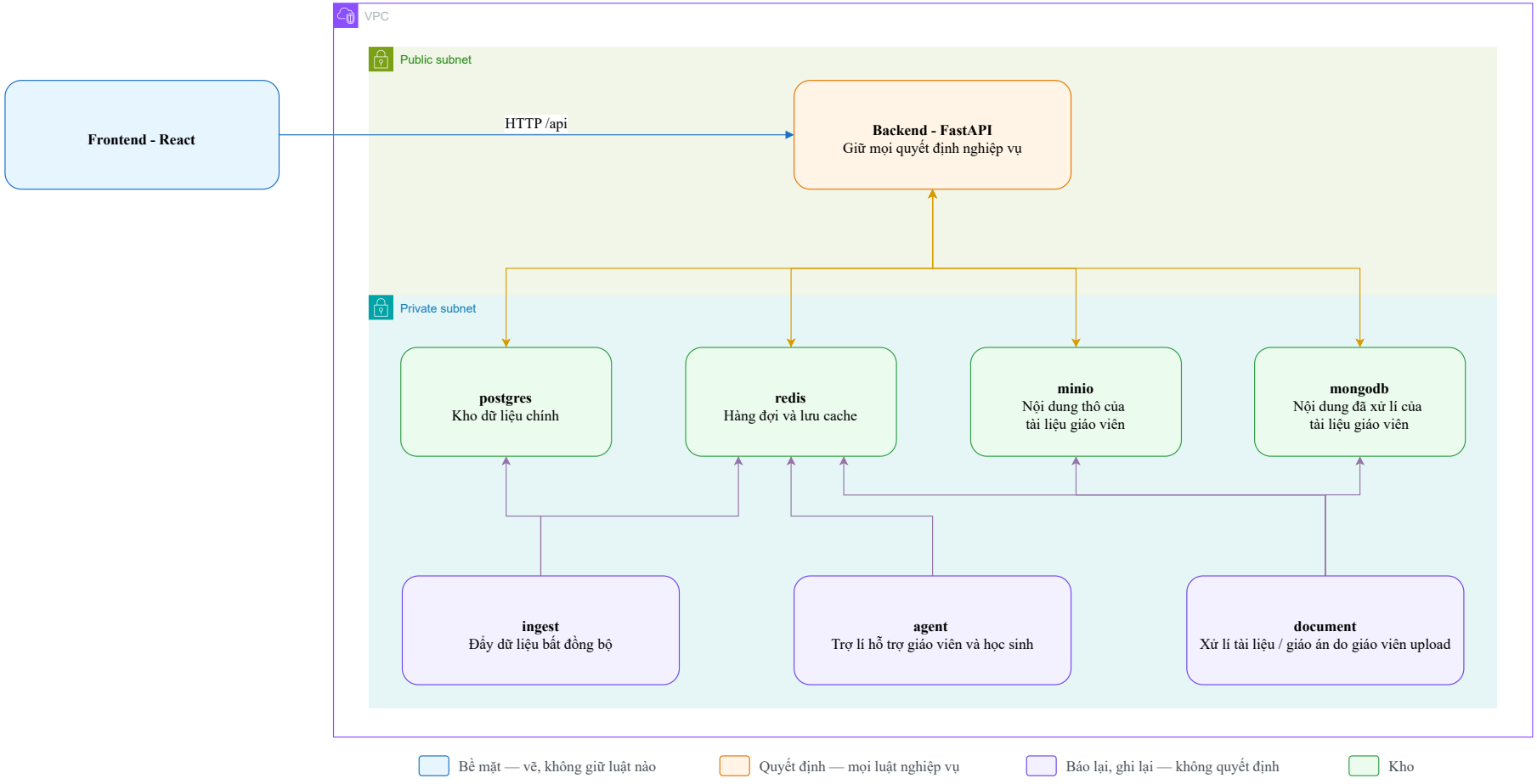
6.1 Các service trong hệ thống

Hệ thống gồm năm **service** (thành phần) chạy độc lập với nhau. Các thuật ngữ tiếng Anh của chương này được định nghĩa ở mục 3.2.

Bảng 6.1: Năm service của hệ thống

Service	Vai trò	Quyết định nghiệp vụ	Credentials
`frontend`	`frontend` cho giáo viên và học sinh. Chỉ nói chuyện với `backend`	Không	Không có
`backend`	Nơi giữ mọi quyết định nghiệp vụ, và là gateway của hệ thống	**Mọi quyết định**	postgres / redis / minio / mongodb
`agent`	Kết nối với Model Provider. Soạn câu hỏi, đề xuất bước tiếp theo cho yêu cầu của giáo viên, và giải đáp học sinh trong hội thoại	Không	redis
`document`	Kết nối với Model Provider. Đọc tài liệu giáo viên tải lên, và dựng cây mục lục cho `agent` dùng khi ra đề	Không	redis / minio / mongodb
`ingest`	Ghi vào database trạng thái xử lí mà `document` báo về	Không	redis / postgres

`frontend` không biết `agent` tồn tại. Mọi thứ nó cần đều đi qua `backend`, nên hệ thống chỉ có một **gateway** (cửa vào duy nhất).



Hình 6.1: System Architecture

6.2 Frontend không nhìn thấy bên trong hệ thống

Hình 6.1 chia hệ thống làm hai vùng, và đặt `frontend` ở ngoài cả hai. Trong **VPC** (đám mây riêng ảo), `backend` nằm ở **public subnet** (vùng mạng công khai); `agent`, `document`, `ingest` và bốn **store** (bốn kho) nằm ở **private subnet** (vùng mạng nội bộ) — chỉ những gì ở public subnet mới nhận được kết nối từ bên ngoài.

`frontend` không bao giờ biết bên trong có mấy service, hay chúng tên gì.

6.3 Backend quyết định mọi thứ

Đây là ràng buộc quan trọng nhất của kiến trúc, và nó là một ràng buộc *nghiệp vụ* chứ không phải một lựa chọn kỹ thuật. Luật chỉ có một câu: **`agent` nộp nội dung / đề xuất hành động, `backend` quyết định làm gì.**

`agent` trả về `backend` hai thứ:

- **Nội dung** — câu hỏi và lời giải cho đề kiểm tra; câu trả lời trong hội thoại với giáo viên hay học sinh; một bản phân tích đọc từ dữ liệu mà `backend` cấp cho nó.
- **Đề xuất hành động** — **tool** (công cụ) nào nên chạy, với tham số nào.

Hai thứ trên có hiệu lực hay không đều do `backend` quyết định. Với nội dung: kiểm trước khi ghi. Với hành động: chỉ chạy những tool chính nó cấp cho mô hình, và từ chối một đề xuất gọi tool ngoài danh sách ấy hay thiếu tham số.

Ví dụ, đối với việc soạn đề:

- `agent` trả về một **đề xuất**: câu hỏi, lời giải, và các phương án lựa chọn, trong đó mỗi phương án nhiễu đi kèm một lỗi cụ thể.
- `backend` kiểm tra lại ràng buộc từng câu rồi mới lưu. Và một đề chỉ rời trạng thái **draft** (nháp) khi giáo viên bấm **approve** (duyệt).

6.4 Service nào kết nối tới store nào?

Bốn store ấy là bốn loại khác nhau: postgres là một **database** (cơ sở dữ liệu) giữ dữ liệu có cấu trúc, redis là một **queue** (hàng đợi), minio là một **object store** (kho tệp) giữ **byte** (dữ liệu gốc) của tệp, và mongodb là một **content store** (kho nội dung) giữ nội dung đã xử lí.

Database thuộc về `backend` và `ingest`. Nó giữ lớp, học sinh, đề, lần làm bài, sổ điểm ba mức, bộ đếm vòng, từng lượt làm lại kèm đề đã sinh ra, các đoạn hội thoại, và các báo cáo.

Object store giữ byte của tài liệu, tức chính tệp giáo viên tải lên. `backend` ghi vào, `document` đọc ra, và không có chiều ngược lại.

Content store giữ thứ đọc được từ tài liệu. Sau khi đọc một tệp, `document` ghi vào nội dung của từng trang và cây mục lục dựng từ chúng. `backend` chỉ đọc ra, để đưa mục lục hay nội dung của một bài cho `agent` khi ra đề theo tài liệu. Tài liệu đã xử lí xong hay chưa thì vẫn do database trả lời: content store giữ *thứ đọc được từ tệp*, không giữ *sự thật về tệp*.

`agent` và `document` không có **credential** (khoá truy cập) của database.

6.5 Các service nói chuyện với nhau thế nào

Hệ thống sử dụng ba hình thức giao tiếp: gọi API trực tiếp, gửi công việc qua hàng đợi và thông báo thay đổi qua Redis Pub/Sub.

Giao tiếp trực tiếp qua API `frontend` gọi `backend`. Đó là đường duy nhất đi vào hệ thống từ bên ngoài, và cũng là lời gọi **API** (giao diện lập trình) duy nhất giữa hai service.

Xử lý công việc qua hàng đợi. Bên gửi đặt một **job** (một đầu việc, một lần chạy) lên queue ở `redis`, kèm **task** là tên của loại việc ấy. Một **worker** (tiến trình nền) ngồi nghe queue đó, lấy job xuống và làm. Bên gửi không bao giờ nói chuyện thẳng với worker; kết quả, nếu cần, cũng đi về qua `redis`.

Thông báo thay đổi qua Pub/Sub. Redis Pub/Sub được sử dụng để thông báo cho các thành phần khác khi có dữ liệu mới hoặc trạng thái thay đổi. Khác với hàng đợi job, thông báo Pub/Sub không được lưu lại để xử lý sau mà chỉ được gửi đến những thành phần đang đăng ký theo dõi channel tại thời điểm phát thông báo.

Ví dụ, khi giáo viên tải lên một tài liệu, `frontend` hiển thị trạng thái "Đang xử lý…", còn `backend` đưa job xử lý tài liệu vào hàng đợi. `document` đọc dữ liệu tệp từ `object store`, trích xuất nội dung và lưu vào `content store`. Sau đó, `document` tạo một job cho `ingest` để cập nhật trạng thái tài liệu trong `database`.

Khi `ingest` hoàn tất việc ghi dữ liệu, nó phát một thông báo lên channel của `redis`. `backend` nhận thông báo này và gửi một event đến `frontend`. Vì event chỉ đóng vai trò báo hiệu, không chứa dữ liệu cập nhật, `frontend` cần gọi lại API để lấy danh sách tài liệu mới nhất và hiển thị trạng thái đã sẵn sàng.

6.6 Job nào đi qua queue, job nào không

Mọi job cần gọi **Model Provider** (nhà cung cấp mô hình) đều chạy qua một queue, đường duy nhất tới `agent` và `document`. Có bảy task như vậy: sáu của `agent`, một của `document`.

Bảng 6.2: Bảy task gọi Model Provider

Task	Service	Khi nào xảy ra
Viết một câu hỏi	`agent`	Mỗi câu của bộ đề giáo viên yêu cầu. **Một câu một job**.
Sinh câu biến thể	`agent`	Khi học sinh nộp bài giai đoạn 1
Trả lời một tin nhắn của học sinh	`agent`	Khi học sinh cần giải đáp thắc mắc ở giai đoạn 2
Đề xuất bước tiếp theo	`agent`	Khi giáo viên gửi một tin nhắn / yêu cầu / thắc mắc
Báo cáo kết quả một plan	`agent`	Sau khi `backend` chạy xong các việc của một plan
Đặt tên một đoạn hội thoại	`agent`	Khi giáo viên gửi tin nhắn đầu tiên và Kriky trả lời
Dựng cây mục lục	`document`	Khi tải lên tài liệu và đã đọc xong nội dung của mọi trang

Chấm bài không đi qua queue. Với trắc nghiệm, chấm là một phép so giữa phương án đã chọn và đáp án đúng đã lưu sẵn, nên nó chạy ngay trong lúc học sinh nộp bài. Kết quả nhìn thấy được trên màn hình: giữa lúc nộp và lúc xem kết quả **không có trạng thái đang chấm nào**.

Màn hình nhận event qua SSE, thay vì phải polling. **SSE** là một kết nối `backend` giữ mở để gửi event xuống khi có gì đổi; **polling** (màn hình tự hỏi lại theo chu kì) là phương án hệ thống không dùng. Có những chỗ dữ liệu hiện ra dần:

- **Hội thoại của học sinh** Hiển thị nội dung phản hồi của trợ lí theo thời gian thực.
- **Hội thoại của giáo viên** Hiển thị nội dung phản hồi của trợ lí, kế hoạch thực hiện (plan) và tiến độ của từng công việc trong kế hoạch theo thời gian thực.
- **Thư viện tài liệu của giáo viên** hiển thị "Đang xử lý…" ngay khi tệp vừa tải lên, rồi tự chuyển sang sẵn sàng mà không cần làm mới màn hình.

6.7 Nếu một service ngừng thì mất gì?

Một kiến trúc chia nhiều phần chỉ đáng giá nếu một phần hỏng không kéo cả hệ thống theo. Bảng 6.3 nói rõ từng trường hợp.

Bảng 6.3: Mỗi service ngừng thì mất gì, còn gì

Ngừng	Mất	Còn
`redis`	Mọi job cần gọi mô hình và tải lên tài liệu	Phát hành đề, làm bài, nộp bài, xem score giai đoạn 1
`minio`	`backend` không khởi động	—
`postgres`	`backend` không khởi động	—
`agent`	Soạn đề, hội thoại	Mọi thứ còn lại
`mongodb`	Ra đề theo tài liệu, và tài liệu mới không xử lí xong	`backend` vẫn khởi động; ra đề không có tài liệu và mọi thứ còn lại
`document`	Tài liệu mới không xử lí xong	Thư viện vẫn mở, tài liệu cũ vẫn dùng được để ra đề
`ingest`	Tài liệu mới không xử lí xong	Job **không mất**: nó nằm lại trên queue, bật lại thì chạy tiếp

6.8 Mô hình dữ liệu

Mô hình dữ liệu có hai mươi hai bảng, chia theo năm nhóm. Mỗi bảng trả lời đúng một câu hỏi nghiệp vụ, và phần ghi chú dưới đây nói vì sao ranh giới giữa chúng nằm ở chỗ đó.

Bảng 6.4: Hai mươi hai bảng, theo năm nhóm

Bảng	Giữ gì
Nhóm 1 — người và lớp	
`classes`	Một lớp giáo viên tạo. Lớp thuộc về giáo viên, không thuộc về nhà trường. Tên lớp *không* phải định danh
`students`	Họ tên, mã học sinh, thuộc lớp nào. Mã học sinh là khoá con người dùng
`teachers`	Họ tên và mã giáo viên
Nhóm 2 — nội dung đề	
`assessments`	Tác giả, tiêu đề, môn, khối, và *trạng thái* — bốn giá trị của vòng đời một đề: rỗng, có câu hỏi, đã duyệt, đã phát hành
`questions`	Đề bài, thứ tự câu học sinh nhìn thấy, mục tiêu học tập
`options`	Nhãn, nội dung, dấu đáp án đúng, và *lỗi mà phương án này đại diện*. Ô lỗi để trống ở đúng một dòng mỗi câu, và dòng đó chính là đáp án đúng
`methods`	Các cách giải của một câu. Bắt buộc nhiều hơn một

tiếp trang sau

Bảng	Giữ gì
`bank_questions`	Ngân hàng câu hỏi của từng giáo viên: bản chép của mỗi câu, kèm phương án và lời giải, lúc đề chứa nó được duyệt. Xoá câu khỏi đề thì bản này ở lại
`publications`	Sáu tham số phát hành, một dòng cho mỗi cặp đề và lớp, nên thu hồi được từng lớp một
`documents`	Tài liệu giáo viên tải lên: tên tệp, kích thước, khoá trỏ tới object store, trạng thái xử lý, số trang, số trang đã đọc, lý do hỏng. **Không giữ byte nào**. Thuộc về giáo viên, không thuộc về đề
`draft_briefs`	Yêu cầu soạn của một đề: môn, khối, phạm vi, tổng số câu đã xin; tài liệu và các nút của cây mục lục khi ra đề theo tài liệu; và các mục dàn ý giáo viên đã đồng ý
`draft_items`	Một ô cho mỗi câu đang soạn, kèm mục dàn ý câu ấy theo, trạng thái, số lần đã thử, và *lý do* lần gần nhất bị loại
Nhóm 3 — bài làm và điểm	
`attempts`	Một học sinh làm một đề: lớp lúc bắt đầu, giờ bắt đầu, giờ phải xong, giờ đã nộp
`answers`	Lựa chọn ở giai đoạn 1, ghi mỗi lần bấm chứ không gom tới lúc nộp
`question_outcomes`	**Sổ điểm**: mark cuối, lý do, số vòng đã dùng, đã chốt hay chưa
Nhóm 4 — giai đoạn 2	
`rounds`	Một lượt làm lại và giờ kết thúc của nó, đã cắt xuống hạn giai đoạn 2 nếu vượt
`round_items`	Câu sinh cho lượt đó, kèm phương án và lời giải — giữ cả đáp án đúng ở đây là thứ cho phép hệ thống tự chấm lượt
`pregenerated_items`	Câu viết *trước* khi học sinh bấm nút, trong lúc em còn đang đọc giải thích
`chat_messages`	Từng tin nhắn giữa học sinh và trợ lí. Câu trả lời hiện dần lên màn hình và chỉ được lưu khi đã trọn vẹn; đóng thẻ trình duyệt giữa chừng thì câu ấy mất, và trợ lí chỉ trả lời lại khi em gửi tin nhắn tiếp theo
`reports`	Báo cáo *giải thích chưa rõ*, gắn với cả bài chứ không với một tin nhắn
Nhóm 5 — hội thoại của giáo viên	

tiếp trang sau

Bảng	Giữ gì
`teacher_conversations`	Một đoạn hội thoại của một giáo viên. Tiêu đề do trợ lí đặt sau lượt hội thoại đầu tiên, giáo viên sửa được; xoá là *xoá mềm*
`teacher_turns`	Từng *bước* của một lượt hội thoại: lời nói, plan, dàn ý, hoặc tool đã chạy, kèm chủ thể mà bước đó nói về

Thứ đọc được từ tài liệu không nằm trong hai mươi hai bảng này mà ở content store; database chỉ biết một tài liệu đã xử lí xong hay chưa.

Chương 7

Chi tiết các service

Chương 6 đã trình bày kiến trúc tổng thể của hệ thống, bao gồm vai trò của các service, phạm vi dữ liệu mà mỗi service quản lý và cách chúng giao tiếp với nhau. Tiếp nối nội dung đó, chương này tập trung phân tích ba service đảm nhiệm các chức năng xử lý chính: `backend`, `agent` và `document`.

7.1 Backend

7.1.1 Vai trò và trách nhiệm

`backend` là **gateway** (cửa vào duy nhất) của hệ thống: mọi yêu cầu từ màn hình của giáo viên hay học sinh đều tới nó qua một lời gọi **API** (giao diện lập trình), và không màn hình nào gọi thẳng một service khác. Với mỗi lời gọi, `backend` xác định người gọi là ai, kiểm dữ liệu được hỏi có thuộc về người ấy không và trạng thái hiện tại có cho phép thao tác ấy không, rồi mới đọc hay ghi **database** (cơ sở dữ liệu). Mọi dữ liệu nghiệp vụ trong database đều do `backend` ghi, trừ kết quả và tiến độ xử lý tài liệu, do `ingest` ghi.

Phần lớn lời gọi được trả lời ngay sau một lần đọc hay ghi database. Ba loại lời gọi thì trả về qua **SSE** (kết nối giữ mở): hội thoại của giáo viên, hội thoại của học sinh và thư viện tài liệu (mục 6.6). Qua kết nối ấy, `backend` gửi từng **event** (sự kiện) xuống màn hình mỗi khi có gì mới.

Những phần xử lý chạy lâu — viết bằng mô hình ngôn ngữ, hay đọc một tệp vừa tải lên — thì `backend` không tự làm. Nó giao chúng đi dưới dạng **job** (một đầu việc, một lần chạy) qua **queue** (hàng đợi) của `redis`, kèm **task** là tên của loại việc ấy: sáu task của `agent` trong bảng 6.2, và một task dò tệp cho `document`, tức xem tệp có sẵn chữ hay là bản chụp; task này không gọi mô hình nên không có trong bảng ấy. `agent` không được ghi vào database, nên mọi thứ nó viết ra đều về qua `backend`, và `backend` quyết định ghi gì. Nhưng `backend` không có **worker** (tiến trình nền) nào: nó chỉ chạy khi có một lời gọi đi qua. Điều này quyết định cách nó lấy kết quả của các job đã giao, trình bày ở mục kế tiếp.

7.1.2 Giao job, lấy kết quả và gửi event

Ba cách lấy kết quả. Không phải job nào cũng cần kết quả ngay, nên `backend` lấy kết quả một job của `agent` theo một trong ba cách, tuỳ việc ai cần kết quả ấy và cần lúc nào (bảng 7.1).

Bảng 7.1: Ba cách `backend` lấy kết quả một job của `agent`, và giới hạn mặc định của mỗi cách

Cách	Dùng cho	Giới hạn mặc định
Lấy kết quả ngay trong lời gọi	Ba task của hội thoại giáo viên; câu biến thể khi phải viết tại chỗ	70 giây cho một job
Chuyển chữ hiện dần, rồi lấy kết quả	Trả lời một tin nhắn của học sinh	25 giây không có chữ mới
Giao job, thu hoạch sau	Viết một câu hỏi của đề; viết trước câu biến thể	Kết quả còn đọc được trong 1 giờ

Cách thứ nhất dùng khi người dùng đang nhìn màn hình và lời gọi chỉ trả lời được khi đã có kết quả, như một bước của lượt hội thoại, hay câu biến thể học sinh cần ngay để làm: `backend` giữ lời gọi mở cho tới khi job xong. Cách thứ hai cũng giữ lời gọi mở, nhưng chuyển từng mảnh chữ `agent` vừa viết xuống màn hình ngay khi nó tới, nên học sinh đọc được câu trả lời trong lúc nó còn đang được viết. Vì vậy giới hạn của cách này đo khoảng im lặng chứ không đo độ dài: mỗi mảnh chữ mới tới thì đồng hồ tính lại từ đầu. Cách thứ ba dùng khi không lời gọi nào cần đứng lấy kết quả của từng job: soạn câu cho đề, mỗi câu một job chạy độc lập, và cả bộ có khi mất vài phút. Câu biến thể cũng được viết trước theo cách này, ngay lúc học sinh nộp bài, để thời gian viết rơi vào lúc em đang đọc giải thích và hỏi trợ lí ở giai đoạn 2. Riêng job dò tệp của `document` không thuộc cách nào: không ai lấy kết quả của nó, vì kết quả đi về qua `ingest` (mục 6.5).

Thu hoạch. Với cách thứ ba, không lời gọi nào đứng lấy kết quả của từng job, và vì `backend` không có worker nên cũng không có tiến trình nền nào nhận thay. Kết quả nằm trong `redis` một giờ rồi mất. Báo cáo gọi việc `backend` lấy những kết quả đã xong ấy về, kiểm từng cái, ghi cái đạt vào database và giao lại cái không đạt là *thu hoạch*.

Thu hoạch chạy theo hai đường. Đường thứ nhất: lời gọi nào đọc hay sửa một đề đang soạn cũng thu hoạch đề ấy trước, như xem chi tiết đề, duyệt đề, xoá câu, soạn thêm hay thay một câu; ở phía học sinh, mở màn hình chữa bài hay mở một lượt làm lại thì thu hoạch câu biến thể viết trước. Đường thứ hai chỉ có khi giáo viên đang xem lượt hội thoại vừa giao việc soạn câu: mỗi khi `agent` báo một câu vừa xong, `backend` thu hoạch ngay rồi gửi số câu mới xuống màn hình.

Hệ quả là một lời gọi chỉ đọc cũng có thể ghi vào database: xem chi tiết một đề có thể đưa thêm câu mới vào đề, còn mở màn hình chữa bài có thể giao thêm job viết trước câu biến thể. Đây là lựa chọn có cân nhắc. Thứ được ghi là kết quả đã có sẵn, và thu hoạch hai lần không sinh thêm gì; còn nếu không thu hoạch, giáo viên sẽ nhìn một đề trống trong khi các câu đã viết xong và đang nằm trong `redis`.

Thông báo và event. `backend` nghe ba loại **channel** (kênh) trên `redis`, theo cơ chế **pub/sub** (phát tin lên kênh, ai đang nghe thì nhận): chữ hiện dần của câu trả lời cho học sinh, thông báo một câu của đề đang soạn vừa xong, và thông báo thư viện tài liệu của một giáo viên vừa đổi. Ở cả ba chỗ, `backend` bắt đầu nghe *trước*, vì pub/sub không giữ lại thứ đã phát. Với chữ hiện dần và câu soạn xong, nó nghe rồi mới giao job; với thư viện tài liệu, nó nghe rồi mới báo màn hình đọc lại danh sách, để lần đọc ấy bù cho thông báo nào đã phát trước lúc bắt đầu nghe.

Event gửi xuống màn hình thì khác nhau tuỳ chỗ. Event của hội thoại giáo viên mang đủ dữ liệu để màn hình vẽ ngay, và mỗi event chỉ được gửi sau khi bước của nó đã ghi vào database, nên tải lại trang vẫn dựng lại được đúng những gì đã hiện. Học sinh nhận từng mảnh chữ, rồi một event kết thúc hoặc một event báo lỗi. Thư viện tài liệu chỉ nhận một event báo có gì vừa đổi, rồi tự gọi lại danh sách. Số câu đã soạn hiện trên màn hình cũng luôn được đếm lại từ database, không cộng dồn từ thông báo, vì một thông báo có thể mất và một vị trí thử lại có thể báo hai lần.

7.1.3 Xử lý các nghiệp vụ chính

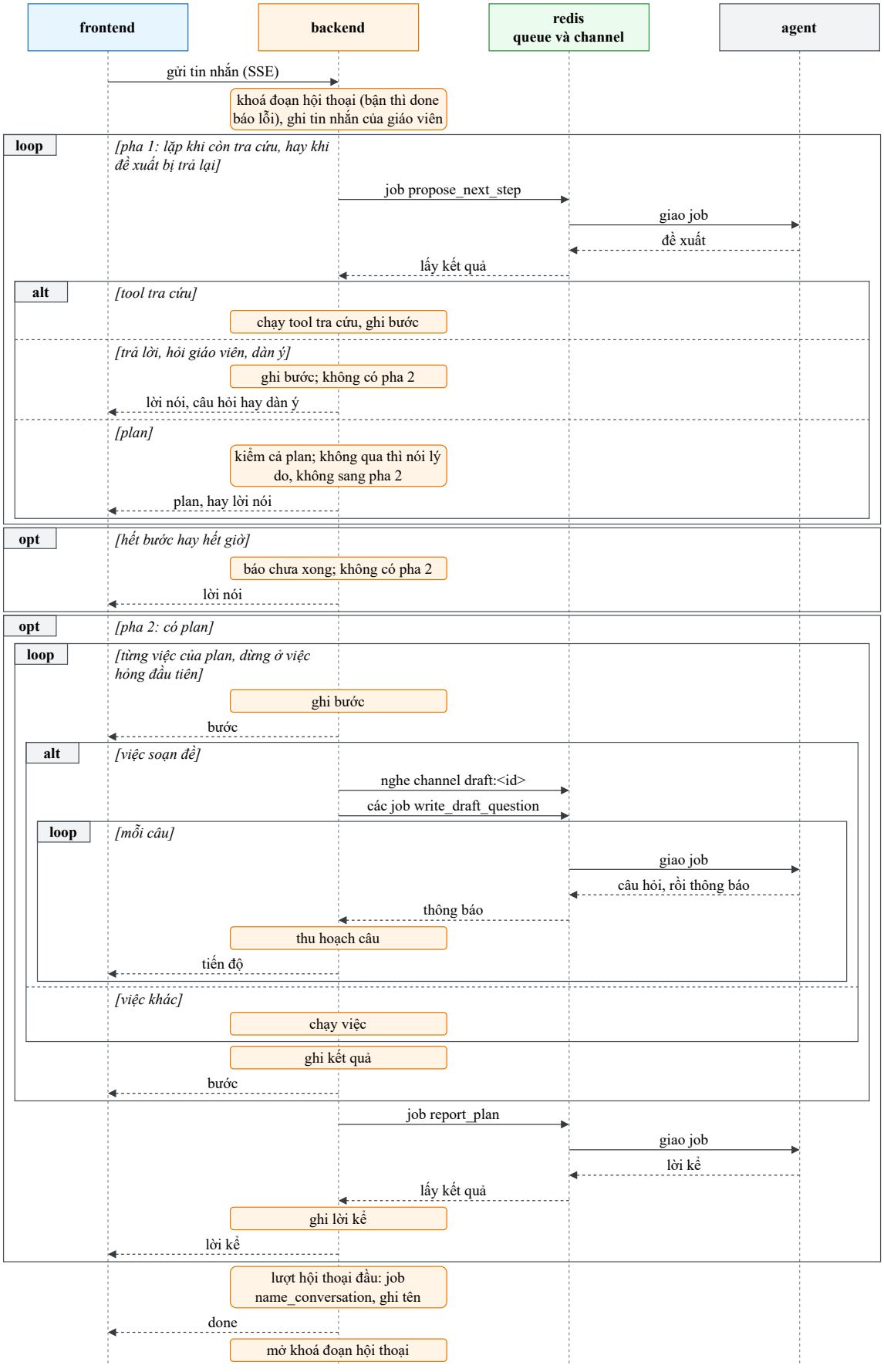
Bốn phần dưới đây đi qua bốn nhóm nghiệp vụ `backend` phụ trách. Luồng nghiệp vụ đã có ở chương 4; ở đây chỉ nói `backend` kiểm gì và quyết định gì trong từng luồng.

Điều phối lượt hội thoại của giáo viên. Hình 7.1 vẽ trọn một lượt hội thoại. Trước khi ghi gì, `backend` khoá đoạn hội thoại bằng một câu lệnh ghi có điều kiện trên database, nên khi hai thẻ trình duyệt cùng gửi thì chỉ một thẻ được chạy; thẻ kia nhận một event báo đoạn hội thoại đang bận. Sau đó lượt hội thoại đi qua hai pha.

Ở pha 1, mỗi bước là một job hỏi `agent` nên làm gì tiếp, và `backend` lấy kết quả ngay trong lời gọi. `agent` có thể xin chạy một **tool** (công cụ) tra cứu, như xem lớp, xem kết quả một đề, đọc mục lục hay chữ của tài liệu; kết quả được đưa lại cho nó ở bước sau. Pha 1 dừng khi `agent` trả lời bằng lời, hỏi lại giáo viên hay đưa ra một dàn ý — khi đó lượt hội thoại kết thúc — hoặc khi nó nêu một **plan** (danh sách việc trợ lí định làm). Pha 1 có hai trần. Trần tám bước chặn một mô hình cứ tra cứu mãi, vì mỗi bước là một lần gọi mô hình phải trả tiền. Trần chín mươi giây cần thêm, vì tám bước mà bước nào cũng chạm giới hạn bảy mươi giây thì dài hơn chín phút; hết chín mươi giây thì pha 1 không mở bước mới. Mỗi phần tài liệu được đọc cộng thêm một bước và mười giây, tối đa mười phần.

Điểm cốt lõi của luồng này là `agent` chỉ đề xuất: không thao tác nào nó nêu tự chạy được. Ở pha 1, `backend` chỉ chạy tool tra cứu; một tool ghi mà `agent` xin chạy ngay bị trả lại kèm lời nhắc rằng tool ấy chỉ nêu được trong plan. Một dàn ý cũng được `backend` kiểm, rồi dựng chữ từ chính các mục và lưu lại trước khi hiện ra, nên thứ giáo viên đọc đúng là thứ các job soạn câu sẽ nhận. Khi `agent` nêu plan, `backend` kiểm plan trước khi việc nào chạy, bằng hai phép kiểm có hậu quả khác nhau. Plan có soạn câu trước hết phải theo đúng dàn ý vừa hiện ra cho giáo viên ở lượt hội thoại ngay trước: số câu, đề và tài liệu mà plan bỏ trống thì `backend` điền từ dàn ý đã lưu, còn plan nêu khác dàn ý, hay chưa có dàn ý nào, thì bị trả lại cho `agent` để sửa ngay trong lượt hội thoại, tốn thêm một bước của pha 1. Sau đó `backend` kiểm *cả* plan: không quá tám việc, vì mỗi việc là một lần ghi và một plan quá dài có thể để lại hàng loạt đề rỗng mang tên giáo viên; mỗi việc gọi một tool ghi có thật và đủ tham số; một việc dùng kết quả của việc khác thì chỉ được trỏ tới việc đứng trước nó. Plan trượt phép kiểm này thì không việc nào chạy, và lượt hội thoại kết thúc bằng một câu xin giáo viên nói lại. Không có tool nào duyệt hay phát hành đề, nên hai thao tác ấy chỉ xảy ra khi giáo viên tự bấm.

Ở pha 2, `backend` tự chạy từng việc của plan theo thứ tự, không hỏi lại `agent` nên làm gì. Kết quả của mỗi việc được ghi vào database rồi mới gửi event, và lượt hội thoại dừng ở việc hỏng đầu tiên. Việc soạn câu đi theo cách thứ ba trong bảng 7.1, và lượt hội thoại nghe thông báo để thu hoạch cho tới khi các câu về đủ. Cuối cùng, `backend` giao một job riêng để `agent` kể lại kết quả, kèm số câu đếm từ database; nếu đoạn hội thoại vừa mở ở lượt này thì thêm một job đặt tên cho nó. Khoá được mở ở cuối, kể cả khi lượt hội thoại hỏng giữa chừng.



Hình 7.1: Một lượt hội thoại của giáo viên, qua SSE

Soạn đề và vòng đời của đề. Một lần soạn mười câu thành mười job, mỗi job một vị trí. Mọi job của một lần soạn mang cùng một bản yêu cầu đã lưu, có số phiên bản: câu nào về mang phiên bản cũ thì bị bỏ, và đề còn câu đang soạn thì không mở được lần soạn mới. Khi thu hoạch một câu, `backend` cho nó qua phép kiểm câu hỏi: đúng một đáp án đúng, các phương án không trùng nhau, mỗi phương án nhiễu gắn một lỗi, và ít nhất hai lời giải; rồi so đề bài với các câu đã có để chống trùng. Câu đạt được đặt vào đúng vị trí giáo viên xin, không theo thứ tự nó về. Câu bị loại thì vị trí ấy được giao lại ngay, kèm đề bài của mọi câu đã có trong đề, kể cả câu vừa ghi, để `agent` tránh, tối đa ba lần thử. Khi thay một câu, câu cũ ở lại trong đề cho tới khi câu mới đạt, rồi hai câu được tráo trong một lần ghi; đề đã có học sinh làm thì không thay câu được.

Duyệt chỉ đi qua khi `backend` đã thu hoạch mọi câu vừa xong, không còn câu nào đang soạn, và đề có ít nhất một câu; trong cùng lần ghi ấy, mọi câu được chép vào ngân hàng câu hỏi. Từ lúc đó, mọi đường ghi câu đều bị từ chối, kể cả thu hoạch, cho tới khi giáo viên bỏ duyệt. Phát hành và bỏ duyệt xử lý nhiều lớp theo hai cách ngược nhau. Một lần phát hành dùng chung một khung giờ cho mọi lớp được chọn nhưng xét từng lớp riêng: lớp nào không phát hành được, như phát hành lại cho một lớp đã qua giờ mở hay đã có học sinh vào làm, thì chỉ lớp ấy trượt. Còn bỏ duyệt một đề đã phát hành là thu hồi nó khỏi mọi lớp, mà mỗi lớp chỉ thu hồi được trước giờ mở của lớp ấy. `backend` xét hết các lớp trước: lớp nào cũng còn trước giờ mở thì thu hồi tất cả trong một lần ghi, chỉ cần một lớp đã qua giờ mở là không lớp nào bị thu hồi.

Làm bài, chấm bài và giai đoạn 2. Giờ phải xong của một **Attempt** (lần làm bài) do `backend` đặt lúc học sinh bắt đầu; đồng hồ trên màn hình chỉ để hiển thị, còn một lựa chọn gửi tới sau giờ ấy bị từ chối. Hai thẻ trình duyệt cùng bấm bắt đầu thì nhận lại cùng một Attempt. Đáp án đúng không tới màn hình học sinh trước khi em nộp: câu gửi xuống không mang dấu đáp án nào, và lời gọi xem lời giải bị từ chối khi chưa nộp. Bài được chấm ngay trong lời gọi nộp (mục 6.6); sau đó `backend` giao job viết trước câu biến thể cho những câu sai, và queue có hỏng thì lần nộp vẫn thành công.

Ở giai đoạn 2, học sinh chỉ mở được một lượt làm lại khi còn trước hạn kết thúc giai đoạn 2 và không có lượt nào đang mở; thời lượng của lượt bị cắt xuống hạn ấy nếu vượt. Câu biến thể được xét từng câu: câu nào đã có câu viết sẵn thì dùng ngay, câu nào chưa có thì `backend` giao một job mới và lấy kết quả ngay trong lời gọi mở lượt, tối đa ba lần thử. Câu biến thể nào lặp lại đề bài của câu gốc hay của các lượt trước đều bị loại. Sau mỗi lượt, điểm của một câu chỉ có thể lên, không xuống. Hội thoại với trợ lí ở giai đoạn này dùng cách thứ hai trong bảng 7.1.

Quản lý tài liệu. Thư viện tài liệu chỉ có ba lời gọi: tải một tệp lên, đọc danh sách kèm trạng thái, và nghe thư viện thay đổi qua SSE. Khi nhận một tệp, `backend` kiểm đuôi tệp, tệp rỗng và cỡ tệp trước khi chạm tới kho nào. Sau đó **byte** (dữ liệu gốc) của tệp được cất vào **object store** (kho tệp) trước, rồi hàng tài liệu mới được ghi vào database; ghi hàng hỏng thì tệp vừa cất bị xoá. Thứ tự này chọn cách hỏng rẻ hơn: một tệp sót lại mà không hàng nào trỏ tới thì không ai thấy, và có thể dọn sau, còn một hàng trỏ vào tệp không có thì hiện lên màn hình như một tài liệu thật. Job dò tệp chỉ được giao sau khi hàng đã ghi xong, để kết quả đọc tệp không về tới trước khi có hàng để ghi vào.

Trạng thái hiện trên màn hình được suy ra lúc đọc: tài liệu đang xử lý mà quá ba trăm giây không báo tiến độ thì hiện là hỏng, dù database vẫn ghi là đang xử lý. Không có lời gọi riêng

nào để màn hình đọc mục lục hay chữ của tài liệu. `backend` đọc chúng từ **content store** (kho nội dung) khi `agent` tra cứu ở pha 1, khi dựng dàn ý và khi mở một đề theo tài liệu; tên chương, bài chỉ lên màn hình qua hội thoại, còn chữ của tài liệu thì không được gửi xuống màn hình. Job soạn câu không nhận chữ nào của tài liệu, chỉ nhận mục dàn ý giáo viên đã xem.

7.1.4 Các ràng buộc và cơ chế kiểm soát

Bảng 7.2 tổng hợp các ràng buộc `backend` giữ, cơ chế thực thi mỗi ràng buộc, và cách kiểm chứng rằng nó vẫn được giữ. Cột cuối có ba loại. *Kiểm thử tự động* là các ca kiểm thử gọi đúng lời gọi hay hàm ấy rồi kiểm kết quả. `check_contract` là một chương trình đọc thẳng mã nguồn và cấu hình của mọi service để bắt những vi phạm mà một ca kiểm thử khó thấy, như tệp khai báo tool gọi tới hàm đổi trạng thái của đề. *Ràng buộc database* thì do chính database từ chối dữ liệu sai.

Bảng 7.2: Ràng buộc `backend` giữ, cơ chế thực thi, và cách kiểm chứng

Ràng buộc	Cơ chế thực thi	Kiểm chứng
Phân quyền		
Dữ liệu của giáo viên khác trả lời như không tồn tại	Mọi truy vấn lọc theo người gọi	kiểm thử tự động
Học sinh không thấy đáp án trước khi nộp	Câu gửi xuống bỏ dấu đáp án; lời giải chỉ mở sau khi nộp	kiểm thử tự động
Không tool nào duyệt, bỏ duyệt, phát hành hay thu hồi đề	Tệp khai báo tool không gọi tới hàm đổi trạng thái	`check_contract`
Ở pha 1, chỉ tool tra cứu chạy được	Danh sách tool chia theo pha	kiểm thử tự động
Tính hợp lệ		
Câu hỏi được kiểm trước khi ghi	`validate_question`; chống trùng khi thu hoạch	kiểm thử tự động
Plan được kiểm trọn trước khi chạy	`vet_plan`	kiểm thử tự động
Câu chỉ soạn theo dàn ý giáo viên vừa thấy	`guard_drafting`	kiểm thử tự động
Job soạn câu không nhận chữ thô của tài liệu	`drafting` không đọc content store	`check_contract`
Tính nhất quán		
Đề đã duyệt từ chối mọi đường ghi câu	Một phép kiểm chung mọi đường ghi câu đi qua, kể cả thu hoạch	kiểm thử tự động

tiếp trang sau

Ràng buộc	Cơ chế thực thi	Kiểm chứng
Chỉ duyệt đề có câu, không còn câu đang soạn; chỉ phát hành đề đã duyệt	Thu hoạch rồi mới đếm câu; phát hành từ chối đề chưa duyệt	kiểm thử tự động
Byte của tệp cất trước, hàng tài liệu ghi sau	Lời gọi tải lên cất tệp trước; ghi hàng hỏng thì xoá tệp	kiểm thử tự động
Điều khiển đồng thời		
Mỗi đoạn hội thoại chỉ chạy một lượt hội thoại một lúc	Câu lệnh ghi có điều kiện	kiểm thử tự động
Mỗi học sinh một Attempt cho một đề; mỗi Attempt chỉ một lượt làm lại chưa nộp	Ràng buộc duy nhất trên `attempts`, `rounds`	ràng buộc database

7.1.5 Xử lý lỗi

Bảng 6.3 nói một service ngừng thì hệ thống mất gì. Bảng 7.3 nói `backend` làm gì khi một thao tác bên trong nó hỏng, chia theo ba nhóm: job và kết quả, kết nối SSE, khoá đoạn hội thoại và tài liệu.

Bảng 7.3: Khi một thao tác bên trong `backend` hỏng

Khi	`backend` làm gì
Job và kết quả	
Không lấy được kết quả trong lúc lời gọi còn mở	Job đề xuất ở pha 1: lượt hội thoại kết thúc bằng một event lỗi. Job kể lại: thay bằng câu `backend` tự dựng từ kết quả các việc. Job đặt tên: lấy câu đầu của giáo viên, đã cắt gọn. Câu biến thể viết tại chỗ: lời gọi mở lượt làm lại báo lỗi tạm thời
Kết quả đã mất khỏi `redis` trước khi được thu hoạch	Câu của đề: giao lại, tính vào ba lần thử. Câu biến thể viết trước: bỏ, rồi giao lại ngay trong lời gọi mở màn hình chữa bài ấy; nếu đang mở lượt làm lại thì viết tại chỗ
Câu không qua phép kiểm, hay trùng câu đã có	Như hàng trên; câu của đề dừng sau ba lần thử, lý do được lưu lại nhưng chưa hiện lên màn hình. Khi thay câu thì câu cũ ở lại
Job soạn câu hỏng hẳn	Vị trí ấy dừng, không giao lại; khi thay câu thì câu cũ ở lại
Kết nối SSE	
Quá lâu không có chữ mới khi đang trả lời học sinh	Báo lỗi qua kết nối SSE; câu trả lời dở không được lưu

tiếp trang sau

Khi	backend làm gì
Lượt hội thoại hỏng hay bị từ chối sau khi kết nối SSE đã mở	Gửi một event kết thúc kèm câu báo lỗi, vì lúc ấy không còn trả được mã lỗi
Channel của đề đang soạn gãy, hay 180 giây không có thông báo	Thôi nghe, thu hoạch một lần cuối, rồi vẫn kể lại kết quả
Khoá đoạn hội thoại và tài liệu	
Đóng thẻ trình duyệt, hay tiến trình chết giữa một lượt hội thoại	Đóng thẻ: khoá vẫn được mở. Tiến trình chết: khoá quá 10 phút thì coi như đã mở
Không giao được job dò tệp	Hàng tài liệu ghi ngay là hỏng, kèm lý do
Tài liệu không báo tiến độ quá 300 giây	Đọc ra là hỏng, mà không ghi gì

7.1.6 Các giới hạn hiện tại

Bốn yêu cầu dưới đây đã được đặt ra cho hệ thống, nhưng `backend` mới thực thi một phần hoặc chưa thực thi; mỗi yêu cầu có một chỗ hở cụ thể:

- **Giờ kết thúc của một lượt làm lại** (chương 4) — lưu từng lựa chọn thì có kiểm giờ, nhưng nộp cả lượt thì không, nên một lượt nộp muộn vẫn được chấm.
- **Câu biến thể giữ cấu trúc của câu gốc** (mục 3.2) — ngoài các phép kiểm chung của mọi câu, chỉ có phép so đề bài với câu gốc và với các lượt trước; không gì so cấu trúc.
- **Plan đã chạy thì chạy hết và được kể lại** — giáo viên đóng thẻ trình duyệt giữa pha 2 thì lượt hội thoại dừng ngay ở việc đang chạy, các việc còn lại không chạy, và không có lời kể nào. Muốn sửa, pha 2 phải chạy thành một job trên queue, mà muốn vậy thì cần một worker — thứ `backend` không có.
- **Giáo viên xem được báo cáo "giải thích chưa rõ" của học sinh** (chương 4) — `backend` mới có đường ghi báo cáo, chưa có lời gọi nào để giáo viên đọc chúng.

7.2 Agent

7.2.1 Vai trò và trách nhiệm

`agent` là service viết những nội dung cần tới mô hình ngôn ngữ: câu hỏi của đề, câu biến thể, câu trả lời cho học sinh ở giai đoạn 2, đề xuất bước tiếp theo trong hội thoại của giáo viên, lời kể lại kết quả một plan, và tên một đoạn hội thoại. Để viết, nó gọi **Model Provider** (nhà cung cấp mô hình). Ngoài Model Provider, nó chỉ gọi thêm một thứ nằm ngoài hệ thống: Jev, mô hình phân loại của TypeSafe AI. Mỗi lần gọi mô hình đều gửi kèm một *lời dặn*, tức đoạn chữ hướng dẫn mô hình phải viết gì; trong hội thoại của giáo viên, Jev chọn phần nào của lời dặn được nạp.

`agent` không nhận lời gọi nào từ bên ngoài và không có quyền vào database. Nó là một worker ngồi nghe queue của mình: `backend` giao job, `agent` viết, rồi đặt kết quả vào `redis` để `backend` lấy về. Vì không tra được database, mỗi job phải tự mang mọi thứ nó cần: job trả lời học sinh mang theo mọi câu em làm sai, kèm các phương án, lời giải, đáp án đúng và phương án em đã chọn, chứ không mang một mã câu hỏi rồi tự đi tra.

Thứ agent trả về chỉ là nội dung, không bao giờ là một quyết định: không điểm, không độ tin cậy, không kết luận học sinh đã hiểu hay chưa. Nó cũng không tự chạy gì: một tool hay một plan nó đề xuất chỉ chạy khi backend chạy nó. Dùng nội dung ấy thế nào, ghi hay bỏ, là việc của backend. Ngoài giá trị trả về, agent còn phát hai thứ qua pub/sub: từng mảnh chữ của câu trả lời cho học sinh, và thông báo mỗi khi viết xong một câu cho đề đang soạn.

7.2.2 Nhận job, gọi mô hình và trả kết quả

agent nhận job của sáu task. Mọi lần nó gọi mô hình để viết chữ đều nằm trong một **graph** (chuỗi hàm cố định) dựng bằng thư viện LangGraph; hai task soạn câu dùng chung một graph. Bảng 7.4 cho biết mỗi task chạy graph nào, gọi mô hình mấy lần và trả về gì; còn khi nào mỗi task được giao thì bảng 6.2 đã nói.

Bảng 7.4: Sáu task của agent: graph, số lần gọi mô hình, và thứ trả về

Task	Graph	Gọi mô hình	Trả về
write_draft_question	authoring	Một tới ba lần	Một câu hỏi
generate_retry_question	authoring	Một tới ba lần	Một câu biến thể
explain_turn	explain	Một lần, trả chữ dần	Câu trả lời cho học sinh
propose_next_step	propose	Một hay hai lần, sau khi Jev chọn tệp lời dặn	Một đề xuất: trả lời, hỏi giáo viên, dàn ý, tool, hay plan
report_plan	reporting	Một lần	Lời kể lại
name_conversation	naming	Một lần	Tên đoạn hội thoại

Graph và hàm. Mỗi hàm của graph làm một phần của job rồi trao kết quả cho hàm sau. Graph trả lời học sinh có hai hàm: hàm đầu dựng lời dặn từ những câu em làm sai, hàm sau gửi lời dặn ấy cho mô hình và nhận chữ về. Graph đề xuất bước tiếp theo cũng có hai hàm: dựng lời dặn, rồi gọi mô hình chọn một đề xuất. Graph soạn câu có hai hàm, viết rồi tự kiểm, nhưng có thêm một đường vòng: hàm tự kiểm thấy câu chưa đạt thì gửi lý do về cho hàm viết để viết một câu khác, và hàm viết chạy tối đa ba lần. Graph kể lại và graph đặt tên chỉ có một hàm. Hàm nhận job, nói ở đoạn dưới, không thuộc graph nào: đó là hàm worker gọi khi lấy một job xuống, và chính nó chạy graph.

Ba lối ra của một job. Mỗi task có một hàm nhận job, và cả sáu hàm đi cùng một đường. Mô hình bị tắt trong cấu hình thì hàm trả *nội dung dọn trước*, tức nội dung mã nguồn tự dựng mà không cần mô hình: một câu viết sẵn, hay vài chữ ghép từ chính job, như sáu chữ đầu của tin nhắn làm tên đoạn hội thoại; mô hình bật thì hàm chạy graph; graph hỏng thì hàm quay về nội dung dọn trước. Thà có một câu dọn trước còn hơn một tính năng từ chối chạy, nhưng lựa chọn ấy có cái giá, nói ở mục 7.2.6. Riêng hàm trả lời học sinh: đã phát được vài chữ rồi mới hỏng thì nó trả về đúng những chữ đã phát làm câu trả lời, không thay bằng câu dọn trước, vì học sinh đã thấy chúng trên màn hình.

Đăng ký task và thông báo. Sáu task được đăng ký dưới đúng tên khai trong gói dùng chung `contracts`, không dưới tên hàm, nên đổi tên một hàm không làm gãy lời giao job của `backend`. Thông báo một câu soạn xong chỉ được phát ngay sau khi kết quả đã ghi vào `redis`: hàm nhận job chỉ đặt sẵn thông báo; worker ghi kết quả vào `redis` khi hàm trả về, rồi mới phát thông báo. Phát sớm hơn thì `backend` nhận thông báo, đi thu hoạch, và chỉ gặp một kết quả chưa có. Thông báo chỉ mang số thứ tự của câu, không mang câu: pub/sub không giữ lại gì, nên thông báo lạc thì câu vẫn nằm trong kết quả của job và `backend` vẫn thu hoạch được. Phát hỏng thì bỏ qua, và job vẫn được tính là xong.

Nối mô hình. Chỉ một tệp, `llm.py`, biết đang gọi Model Provider nào, nên đổi Model Provider chỉ là sửa cấu hình; Jev thì đi qua một tệp riêng. `llm.py` đặt trần thời gian mặc định hai mươi giây cho mỗi lần gửi tới Model Provider, và xếp một mô hình dự phòng sau mô hình chính nếu cấu hình có. Hai graph soạn câu và đề xuất bước tiếp theo bắt mô hình trả lời theo một khuôn JSON cố định, tức một bộ ô có tên sẵn mà mô hình phải điền, như đề bài, các phương án, lời giải; ba graph còn lại nhận chữ thường. Chữ trong khuôn — câu hỏi soạn ra, lời nói của một đề xuất, tên các việc của plan, dàn ý — còn đi qua một bộ dựng lại ký hiệu toán. Mô hình viết toán bằng LaTeX, mà trong một chuỗi JSON, dấu gạch chéo đứng trước f, n, r, t hay b là mã của một ký tự điều khiển, nên `\frac` về tới nơi thành một ký tự điều khiển cộng với "rac". Lỗi này đã lọt vào database thật ngày 05/10/2026. Bộ dựng lại đi theo một danh sách lệnh LaTeX đã biết, nên một lệnh không có trong danh sách thì để nguyên, chứ không bị đoán sai.

7.2.3 Xử lý các task chính

Bốn phần dưới đây đi qua bốn nhóm task. Mỗi phần nói `agent` nhận gì, viết gì, và thứ nó viết được kiểm ở đâu, nếu có; `backend` làm gì với kết quả thì mục 7.1 đã nói.

Soạn câu hỏi và câu biến thể. Hai task này dùng chung graph soạn câu, chỉ khác lời dặn. Job soạn một câu của đề mang mục dàn ý sinh ra câu ấy: kiến thức cần kiểm, công thức và cách làm, đúng như giáo viên đã đọc. Các job của một lần soạn chạy song song và không thấy nhau, nên mỗi job còn được biết mình là câu thứ mấy và kiến thức của các câu khác để khỏi hỏi sang, và mang đề bài của những câu đã có trong đề để phép tự kiểm loại một câu lặp lại; nếu vẫn trùng thì `backend` bắt khi thu hoạch. Job câu biến thể mang câu gốc, phương án học sinh đã chọn và lỗi em mắc, cùng đề bài của các lượt trước; đề bài câu gốc và các lượt ấy đều bị cấm lặp. Lời dặn đòi một câu cùng dạng và cùng cách làm nhưng khác số liệu, có một phương án nhiễu ứng với đúng lỗi ấy. Viết xong, câu qua hàm tự kiểm, xét bốn điều: đúng một đáp án đúng, mọi phương án nhiễu gắn một lỗi, ít nhất hai lời giải, và đề bài không trùng một đề bài bị cấm. Câu không đạt thì lý do được gửi về cho hàm viết để viết một câu khác; sau ba lần vẫn không đạt thì hàm nhận job trả nội dung dọn trước. Phép tự kiểm này không thay phép kiểm của `backend`, vốn rộng hơn; nó chỉ để một câu sai hình dạng không phải đi tới `backend` rồi bị giao lại.

Trả lời học sinh. Mỗi tin nhắn của học sinh là một job và một lần gọi mô hình. Lời dặn yêu cầu trợ lí giải thích chứ không kết luận — không cho điểm, không nói em đã hiểu — và dùng đúng nhãn lỗi đã gắn sẵn cho phương án em chọn từ lúc soạn đề thay vì tự chẩn đoán lại, để hai học sinh mắc cùng một lỗi được giải thích theo cùng lỗi ấy. Trong lúc mô hình viết, hàm nhận job phát từng mảnh chữ lên channel mà `backend` đang nghe; khi job xong, cả câu trả lời vẫn được trả về, và đó mới là thứ được lưu.

Đề xuất bước tiếp theo. Đây là task phức tạp nhất: graph đề xuất chiếm hơn một phần ba mã nguồn và ít nhất một nửa số ca kiểm thử của `agent`. Mỗi bước của pha 1 trong hình 7.1 là một job của task này, và đề xuất của nó đi qua ba tầng, mỗi tầng bắt một loại sai khác.

Tầng một là chính mô hình, đề xuất trong một khuôn. Lời dặn của task này không viết liền một khối: phần chỉ một loại việc cần nằm trong ba *tệp lời dặn*, một cho việc tra lớp và kết quả, một cho việc soạn đề, và một tệp chung. Trước khi graph chạy, Jev đọc bốn tin nhắn gần nhất và trả lời có hay không cho từng tệp chuyên biệt; không tệp nào được chọn thì nạp tệp chung, còn Jev không trả lời được thì nạp cả ba. Lời dặn gửi mô hình ghép từ một lõi chung, các tệp vừa chọn, tên giáo viên, tài liệu đang đính kèm, và hai danh sách tool: tool gọi được ngay, và tool chỉ nêu được trong plan. Những luật mà lượt hội thoại nào cũng cần nằm trong lõi, vì Jev chọn sai thì một tệp vắng mặt. Mô hình trả về đúng một trong năm loại đề xuất: trả lời, hỏi giáo viên, đưa ra dàn ý, gọi một tool, hay nêu plan. Khuôn chỉ có những ô có tên sẵn, nên một đề xuất sai hình dạng không qua được tầng này.

Khuôn chỉ hỏi những gì mô hình trả lời được, vì mỗi ô là một câu hỏi đặt cho nó, và không ô nào được để ngỏ. Bài học ấy có giá: khuôn đầu tiên là nguyên kiểu dữ liệu `agent` trả cho `backend`, nên hỏi cả những thứ mô hình không thể biết, như mã của job, và có một ô tham số nhận bất cứ thứ gì. Model Provider từ chối thẳng mọi lần gọi vì ô ấy, và nội dung dọn trước che kín chuyện đó sau một câu trả lời nghe hợp lý.

Tầng hai là tám luật trong `agent`, sửa đề xuất ngay sau khi mô hình trả lời. Sáu luật giữ thứ tự tra cứu, phần lớn quanh tài liệu: plan mở đầu bằng một tool tra cứu thì tool ấy được gọi trước, chưa đọc mục lục thì chưa được nêu plan soạn theo tài liệu, chưa đọc chữ của phần ấy thì chưa được đưa ra dàn ý. Một đề xuất đến sớm như vậy thành một lời gọi tool tra cứu, với tham số lấy từ chính đề xuất hay từ những gì đã có trong hội thoại, không đoán thêm; ở bước sau, mô hình nêu lại với kết quả trong tay. Luật thứ bảy sửa lời một câu hỏi lại đứng ngay sau danh sách tài liệu, cho khớp các nút giáo viên sẽ bấm. Luật thứ tám sửa mục đích của một dàn ý: giáo viên nói rõ là thay câu thứ ba mà dàn ý lại viết cho cả một đề mới thì dàn ý được thu về đúng mục chứa câu ấy. Riêng khi giáo viên vừa được xem dàn ý và trả lời bằng một câu chỉ gồm lời đồng ý, như "ok" hay "được", thì `agent` dựng plan thẳng từ dàn ý và bỏ đề xuất của mô hình, vì sau một chữ "ok" mô hình hay quên phạm vi vừa bàn.

Sau các luật sửa là các phép kiểm: plan thiếu tham số bắt buộc, đúng luật `backend` dùng khi kiểm plan, hay có ký tự điều khiển trong tham số; dàn ý rỗng, hay có tổng số câu khác số câu mô hình vừa ghi nhận; câu hỏi lại giáo viên có quá ít lựa chọn; hay một đề xuất không có chữ nào. Đề xuất vấp một phép kiểm thì `agent` chạy lại graph thêm đúng một lần. Lần hai vẫn vấp thì nó giữ lại một trong hai đề xuất để `backend` xét, chứ không tự bịa tham số hay lựa chọn; hai lần đều rỗng thì nó trả một câu trung tính nhờ giáo viên nói lại.

Tầng ba là `backend`: đề xuất về tới đó vẫn chỉ là đề xuất, và đoạn *Điều phối lượt hội thoại của giáo viên* ở mục 7.1 đã kể nó được kiểm những gì.

Kể lại và đặt tên. Lời kể lại kết quả một plan là một job riêng, không phải một vòng nữa của task đề xuất: nó đọc kết quả của các việc đã chạy và không cầm danh sách tool nào, nên không thể đề xuất thêm việc. Số câu đã soạn được đưa vào như một dòng dữ liệu, dựng từ con số `backend` đếm trong database; lời dặn cấm nói quá điều đã làm (chỗ hở của luật này ở mục 7.2.6). Đặt tên cho một đoạn hội thoại là task nhỏ nhất: một hàm, nhận câu mở đầu của giáo viên và trả về một cái tên ngắn. Ở đây `backend` có phép kiểm thật: cái tên phải có ít nhất một từ chung với câu mở đầu, không thì `backend` dùng chính câu mở đầu, đã làm gọn.

7.2.4 Các ràng buộc và cơ chế kiểm soát

Bảng 7.5 tổng hợp các ràng buộc `agent` giữ, cơ chế thực thi mỗi ràng buộc, và cách kiểm chứng. Cột cuối dùng hai trong ba loại đã giải thích trước bảng 7.2; không có ràng buộc database, vì `agent` không có database.

Bảng 7.5: Ràng buộc `agent` giữ, cơ chế thực thi, và cách kiểm chứng

Ràng buộc	Cơ chế thực thi	Kiểm chứng
Phân quyền		
Kết quả chỉ mang nội dung, không mang quyết định nào, kể cả trong các trường lồng nhau	Kiểu dữ liệu trả về của mọi task chỉ có nội dung	kiểm thử tự động
Không có quyền vào database	Mã nguồn không nhắc tới thư viện hay thông tin kết nối database nào	`check_contract`
Lời kể lại không cầm tool nào	Yêu cầu của job kể lại không có trường nào mang tool	`check_contract`
Tính hợp lệ		
Câu hỏi đúng hình dạng	Phép tự kiểm trong graph soạn câu; `backend` kiểm lại khi thu hoạch	kiểm thử tự động
Chữ trong khuôn đi qua bộ dựng lại ký hiệu toán	`latex_escapes`, ở graph soạn câu và graph đề xuất	`check_contract`
Tính nhất quán		
Task đăng ký dưới đúng tên trong `contracts`	Danh sách task của worker	kiểm thử tự động
Thông báo câu soạn xong phát sau khi kết quả đã ghi, chỉ mang số thứ tự	Worker phát thông báo khi job đã kết thúc	kiểm thử tự động
Câu trả lời bị cắt giữa chừng giữ đúng những chữ đã phát	Hàm nhận job của task trả lời học sinh	kiểm thử tự động
Thời gian		
Một job, kể cả lời chọn của Jev và các lần graph gọi lại mô hình, kết thúc trước khi `backend` thôi lấy kết quả	So cấu hình hai service: Jev 3 giây + 20 giây × 3 lần gọi = 63 giây, dưới 70 giây	`check_contract`

Hai ràng buộc trong bảng được kiểm chứng hẹp hơn lời của chúng. Ca kiểm thử "không mang quyết định" chạy mọi task hai lần, với mô hình tắt rồi bật, nhưng mô hình giả dùng khi kiểm thử không dựng được đầu ra theo khuôn: ở lần chạy bật, ba trong sáu task vẫn rơi về nội dung dọn trước, nên đầu ra thật của graph soạn câu và graph đề xuất chưa bị soi. Còn phép kiểm thời gian coi hai mươi giây là trần của cả một lần gọi, trong khi đó chỉ là trần của một lần

gửi: thư viện của OpenAI tự gửi lại tới hai lần khi quá giờ, rồi mới tới mô hình dự phòng. Một lần gọi vì thế có thể dài hơn sáu mươi giây, và `backend` có thể thôi lấy kết quả trong lúc worker vẫn đang viết.

7.2.5 Xử lý lỗi

Bảng 7.6 nói `agent` làm gì khi mô hình hay Jev hỏng, hay khi thứ mô hình viết vấp phép kiểm, theo từng nhóm task. Đường lùi chung là nội dung dọn trước, như đoạn *Ba lối ra của một job* đã nói; bảng cho thấy nó trông thế nào ở từng task.

Bảng 7.6: Khi một thao tác bên trong `agent` hỏng

Khi	`agent` làm gì
Mọi task	
Mô hình quá giờ hay báo lỗi	Gọi mô hình dự phòng nếu cấu hình có; vẫn hỏng thì xử lý theo từng task dưới đây
Soạn câu hỏi và câu biến thể	
Câu vấp phép tự kiểm	Gửi lý do về để viết lại; hàm viết chạy tối đa ba lần, vẫn vấp thì xử lý như mô hình hỏng
Mô hình hỏng khi soạn một câu của đề	Trả một câu dọn trước, lấy từ sáu câu Toán cố định
Mô hình hỏng khi viết câu biến thể	Câu gốc thuộc bộ câu dọn trước và còn biến thể chưa dùng thì trả biến thể ấy; không thì trả lại chính câu gốc, thêm đuôi “(đề thay số cho lượt …)”
Phát thông báo câu soạn xong hỏng	Bỏ qua; job vẫn xong, và `backend` vẫn thu hoạch được câu
Trả lời học sinh	
Mô hình hỏng trước khi phát chữ nào	Trả một câu trả lời dọn trước
Mô hình hỏng giữa chừng	Trả đúng những chữ đã phát làm câu trả lời
Đề xuất bước tiếp theo	
Jev chưa được cấu hình, báo lỗi, quá 3 giây, hay trả lời một hình dạng lạ	Nạp cả ba tệp lời dặn
Đề xuất vấp phép kiểm	Chạy lại graph đúng một lần và giữ đề xuất lần hai, trừ ba trường hợp: lần đầu có đề xuất mà lần hai rỗng hay mô hình hỏng thì giữ lần đầu; cả hai là câu hỏi quá ít lựa chọn thì giữ lần nhiều lựa chọn hơn; cả hai đều rỗng thì trả một câu trung tính nhờ giáo viên nói lại

tiếp trang sau

Khi	agent làm gì
Mô hình hỏng	Trả một đề xuất dọn trước, đoán việc theo vài chữ trong tin nhắn
Kể lại và đặt tên	
Mô hình hỏng khi kể lại	Ghép một câu từ kết quả các việc
Mô hình hỏng khi đặt tên	Lấy sáu chữ đầu của câu mở đầu

7.2.6 Các giới hạn hiện tại

Năm yêu cầu dưới đây đã được đặt ra cho hệ thống, nhưng agent mới thực thi một phần hoặc chưa thực thi. Ba yêu cầu đầu hở vì cùng một lý do: nội dung dọn trước không mang dấu nào để backend nhận ra nó. Mỗi yêu cầu có một chỗ hở cụ thể:

- **Câu hỏi theo đúng môn và phạm vi giáo viên xin** — mô hình hỏng khi soạn một câu thì câu dọn trước là một trong sáu câu Toán cố định, bất kể môn hay phạm vi. Câu ấy đúng hình dạng nên qua phép kiểm của backend; chỉ lúc giáo viên duyệt đề mới chặn được nó.
- **Câu biến thể khác câu gốc** — khi mô hình bật, câu gốc do mô hình viết nên không thuộc bộ câu dọn trước, và mô hình hỏng thì học sinh nhận lại đúng câu gốc kèm một đuôi nói số đã đổi. Phép so đề bài của backend coi đuôi ấy là một đề bài khác, nên không cổng nào chặn.
- **Đề xuất đúng việc giáo viên nhờ** — mô hình hỏng thì đề xuất dọn trước đoán việc theo vài chữ trong tin nhắn, và có thể đoán sai: "ra đề Toán lớp 10 về mệnh đề" thành câu hỏi "Bạn muốn xem lớp nào?". Phép đoán không còn tự nêu plan soạn đề: nó chỉ nêu plan khi giáo viên vừa đồng ý một dàn ý. Nhưng nó vẫn đưa ra được một dàn ý tự dựng, và backend không biết một đề xuất đến từ mô hình hay từ phép đoán.
- **Lời kể lại không nói quá điều đã làm** — chỉ lời dặn canh điều này. Con số trên màn hình do backend tự đếm nên không sai theo, nhưng câu mô hình viết đi thẳng lên màn hình mà không ai so lại.
- **Giáo viên đồng ý dàn ý trước khi câu được soạn** — luật trong agent chỉ bắt câu chỉ gồm lời đồng ý; câu khác thì đề xuất của mô hình đi nguyên, và mô hình có thể đọc "ok nhưng đổi câu 2" thành đồng ý rồi nêu plan soạn câu. backend chỉ chứng minh được dàn ý đã hiện ra cho giáo viên, nên plan ấy vẫn đi qua.

7.3 Document

7.3.1 Vai trò và trách nhiệm

document đọc những tệp giáo viên tải lên và dựng cây mục lục cho chúng; một tài liệu chỉ dùng được khi có đủ cả hai. Service này gồm hai worker, không nhận lời gọi nào từ bên ngoài và không có quyền vào database. Job tới mang địa chỉ của tệp trong object store, không mang byte của tệp; worker tự lấy byte về để đọc.

Mọi thứ document đọc ra và dựng nên đều nằm ở content store: chữ của từng trang, gọi là *bản thô*, cây mục lục, và các **chunk** (mẩu chữ ngắn) dưới từng nút của cây, chia vào sáu **collection** (nhóm bản ghi cùng loại, như một bảng của database). Còn kết quả xử lý và tiến

độ thì `document` không ghi vào database: nó gửi chúng đi thành job trên queue của `ingest`, và `ingest` mới là bên ghi.

Để dựng cây, `document` gọi hai thứ nằm ngoài hệ thống, cũng là hai thứ `agent` gọi: Model Provider, để đọc tên chương trên ảnh và tóm tắt từng nút của cây, và Jev, để quyết mỗi dòng có dáng tiêu đề thật ra là gì và thuộc cấp nào của cây. Mặc định, `gpt-5.6-luna` đọc tên chương và `gpt-4o-mini` viết tóm tắt; mỗi mô hình đổi được sang một mô hình Claude bằng cấu hình. Còn **OCR** (đọc chữ từ ảnh) thì không gọi ra ngoài: ba mô hình của nó chạy ngay trong worker đọc tệp.

7.3.2 Hai worker, ba task và sáu collection

`document` nhận job của ba task, và hai trong số đó nó tự giao cho mình (bảng 7.7): một bản chụp được đọc thành nhiều *lô*, mỗi lô tối đa mười trang và là một job `ocr_pages`, rồi cây được dựng trong một job `index_document`. Ra khỏi service thì nó chỉ gửi job của hai task, cả hai lên queue của `ingest`: `document_progress` báo tiến độ, `document_probed` mang kết quả cuối cùng của một tệp.

Bảng 7.7: Ba task của `document`: worker nào nhận, ai giao, và job gửi đi

Task	Worker nhận	Ai giao	Gửi đi
`probe_document`	Đọc tệp	`backend`, khi giáo viên tải tệp lên	Tệp có sẵn chữ: `index_document`, sau khi đã ghi bản thô. Bản chụp: lô `ocr_pages` đầu tiên. Tệp hỏng hay rỗng: `document_probed` kèm lý do
`ocr_pages`	Đọc tệp	Chính worker đọc tệp	`document_progress` sau mỗi lô trừ lô cuối; rồi lô kế, hay sau lô cuối `index_document` nếu cả cuốn có chữ; `document_probed` khi cả cuốn không ra chữ nào, hay một trang không đọc nổi
`index_document`	Dựng cây	Worker đọc tệp	`document_progress` trong lúc dựng; `document_probed` khi cây đã ghi xong, hay khi dựng hỏng

Vì sao có hai worker. Đọc tệp tốn bộ xử lý, còn dựng cây tốn thời gian gọi Jev và Model Provider qua mạng. Worker đọc tệp vì vậy chạy một job một lúc: OCR chiếm trọn bộ xử lý, nên hai lô chạy song song chỉ tranh nhau rồi cùng chậm. Nó nạp ba mô hình OCR một lần lúc khởi động, mất năm tới sáu giây, vì nạp lại ở mỗi lô là tốn thêm năm, sáu giây ấy cho từng lô của một cuốn. Worker dựng cây không nạp mô hình nào và chạy tới bốn job song song. Gộp làm một worker thì phải chọn: chạy một job một lúc thì job dựng cây xếp sau mọi lô OCR, chạy nhiều job thì các lô OCR tranh bộ xử lý. Cái giá của việc chỉ chạy một job một lúc nằm ở mục 7.3.6.

Tự giao job và báo tiến độ. Job của một lô OCR mang một mã cố định ghép từ tài liệu và trang đầu của lô; job dựng cây mang một mã cố định theo tài liệu. Một lô bị chạy lại sau khi đã giao lô kế thì lần giao thứ hai bị từ chối, nên một cuốn không có hai chuỗi lô đọc song

song, cũng không có hai job dựng cùng một cây. Mã chỉ chặn trùng khi job kia còn trên queue hay đang chạy, nên điều này chỉ đúng khi có một worker đọc tệp, như hiện nay. Worker dựng cây báo tiến độ ngay trong lúc dựng — sau khi nạp bản thô, sau mỗi tên chương đọc được, sau khi Jev xếp xong, và trong lúc tóm tắt — để `backend` không đọc một cuốn đang dựng ra là hỏng sau ba trăm giây; dòng trạng thái khi ấy hiện "Đang dựng mục lục…".

Sáu collection. Bảng 7.8 kể sáu collection, ai ghi và ai đọc. Chỉ `document` ghi content store; `backend` chỉ đọc cây và các chunk, khi hội thoại của giáo viên cần tới.

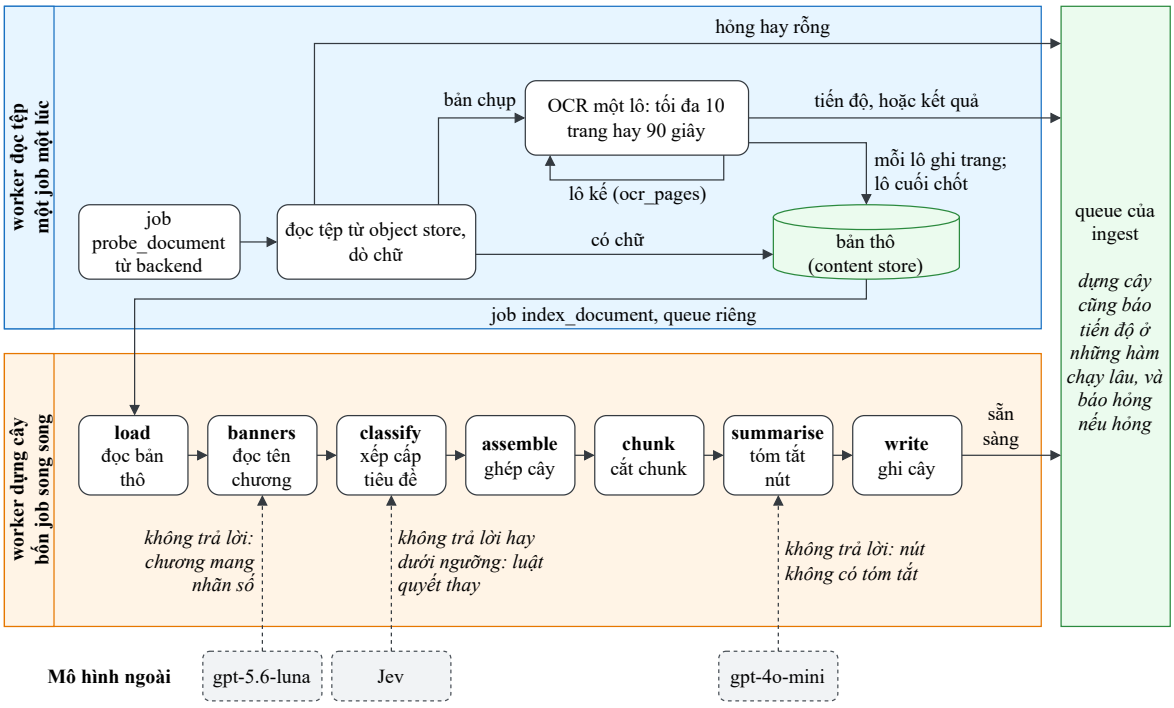
Bảng 7.8: Sáu collection của content store: giữ gì, ai ghi, ai đọc

Collection	Giữ gì	Ai ghi	Ai đọc
`documents`	Bản ghi chung của bản thô, khác bảng cùng tên của database: đọc thẳng chữ có sẵn hay qua OCR, đã đọc xong chưa	Worker đọc tệp	Worker dựng cây
`pages`	Chữ của từng trang, chia theo khối	Worker đọc tệp	Worker đọc tệp, để bỏ qua trang đã đọc; worker dựng cây
`indexes`	Bản ghi chung của cây (số nút, số chunk, cách xếp), ghi sau cùng	Worker dựng cây	Worker dựng cây, `backend`
`nodes`	Các nút của cây mục lục	Worker dựng cây	`backend`
`chunks`	Các chunk của mỗi nút	Worker dựng cây	`backend`
`block_labels`	Quyết định cho từng dòng có dáng tiêu đề: ai quyết, Jev đã phân vân giữa những gì; để gỡ lỗi và đo	Worker dựng cây	Không ai, lúc chạy

7.3.3 Một tệp đi qua service

Hình 7.2 vẽ đường đi của một tệp qua hai worker, từ lúc worker đọc tệp lấy job dò tệp xuống tới lúc kết quả sẵn sàng rời service.

Dò tệp. Worker đọc tệp lấy byte của tệp từ object store rồi dò xem tệp có sẵn chữ không. Tệp `.txt` và `.md` phải là UTF-8, hay UTF-16 có BOM; bảng mã khác bị từ chối chứ không đoán, vì đoán sai thì ra chữ trông như chữ mà sai dấu. Với tệp PDF, nó đếm số trang có chữ, và một trang tính là có chữ khi có từ một trăm ký tự trở lên: từ mười phần trăm số trang có chữ trở lên thì tệp có sẵn chữ, còn lại là bản chụp. Một PDF không có trang nào được báo là không có chữ, không đi OCR. Đếm trang chứ không lấy trung bình ký tự là có chủ ý: một bản chụp dày mà riêng trang mục lục có sẵn chữ, đủ nhiều để kéo trung bình lên quá một trăm ký tự một trang, vẫn được nhận ra là bản chụp.



Hình 7.2: Một tệp đi qua `document`: hai worker, ba mô hình ngoài, và các đường lùi

Bản chụp đọc theo lô. Một bản chụp được đọc qua nhiều job, mỗi job một lô: tối đa mười trang, và lô đã chạy quá chín mươi giây thì không mở trang mới, để lô không chạm trần một trăm hai mươi giây của worker. Mỗi lô ghi trang của nó vào content store; một lô bị chạy lại thì chỉ đọc những trang chưa có trong content store. Mỗi trang đi qua năm *chặng*, tức năm công đoạn nối tiếp nhau của OCR: dựng ảnh, nhận khối, tìm dòng, đọc chữ từng dòng, rồi gán dòng vào khối. Hai chặng nhận khối và tìm dòng dùng mô hình của PaddleOCR, còn chặng đọc chữ dùng một bộ đọc khác, mặc định là EasyOCR, vì bộ đọc chữ của PaddleOCR không phát ra một số chữ tiếng Việt mang hai dấu, như "ế" hay "ộ". Công thức toán không được đọc riêng. Trên một bộ đo năm mươi trang có năm mươi tám công thức, mô hình đọc công thức từng được thử chỉ đọc đúng chín: hai mươi bốn công thức đã hỏng từ chặng nhận khối, nên mô hình nào đứng sau cũng không cứu được, mô hình ấy đọc sai thêm hai mươi mốt, và bốn công thức còn lại hỏng mà chưa rõ do đâu. Khối công thức vì vậy chỉ nhận chữ OCR như mọi khối khác.

Bản thô ghi trước. Đọc xong, bản thô của tệp, chia theo trang và theo khối, đã nằm trong content store. Tài liệu lúc này *chưa* sẵn sàng, vì nó còn cần cây mục lục. Worker đọc tệp giao một job `index_document` lên queue riêng của worker dựng cây, và chưa gửi kết quả nào về `ingest`.

Bảy hàm dựng cây. Job dựng cây chạy một graph gồm bảy hàm nối thẳng (bảng 7.9). Mô hình không tự dựng cây: hàm `assemble` ghép cây bằng mã nguồn, từ quyết định của hàm `classify`. Mô hình chỉ góp phần vào ba hàm, và cả ba đều có đường lùi ngay bên trong hàm: mô hình không trả lời, hay chưa được cấu hình, thì tài liệu vẫn sẵn sàng, chỉ kém hơn.

Ở hàm `classify`, luật và Jev cùng đưa ra một quyết định cho mỗi dòng có dáng tiêu đề, và Jev chỉ thắng khi độ tin cậy của nó từ tám mươi phần trăm trở lên, một ngưỡng cấu hình được. Jev được hỏi riêng cho từng dòng, và chỉ một lời gọi hỏng là luật quyết cả cây, để cây không

Bảng 7.9: Bảy hàm dựng cây mục lục, và đường lùi của những hàm có gọi mô hình

Hàm	Làm gì	Mô hình không trả lời thì
`load`	Đọc bản thô; bản thô chưa đọc xong thì dừng cả job	—
`banners`	Chỉ với bản chụp: đọc tên chương trên ảnh rộng ở đầu trang, khi chữ OCR trong ảnh có CHƯƠNG hay PHẦN, bằng Model Provider (mặc định `gpt-5.6-luna`)	Chương mang nhãn số
`classify`	Xếp vai trò và cấp cho từng dòng có dáng tiêu đề; luật và Jev cùng quyết, Jev thắng khi đủ chắc	Luật quyết thay
`assemble`	Ghép cây tối đa ba cấp, không nhảy cấp; tiêu đề sâu hơn gộp vào nút cha	—
`chunk`	Cắt chữ thành các chunk dưới từng nút	—
`summarise`	Tóm tắt từng nút, từ cấp sâu nhất lên; nút cha tóm từ tóm tắt của các con, bằng Model Provider (mặc định `gpt-4o-mini`)	Nút không có tóm tắt
`write`	Ghi cây vào content store	—

nửa Jev nửa luật. Tóm tắt của mỗi nút là để trợ lí, khi đọc mục lục, chọn đúng chương hay bài chỉ bằng tiêu đề và tóm tắt, chưa phải đọc chữ của nút. Vì sao chưa nói được Jev đã làm cây tốt hơn hay chưa thì nằm ở mục 7.3.6.

Ghi cây rồi mới báo sẵn sàng. Hàm cuối ghi các nút, các chunk và quyết định cho từng dòng có dáng tiêu đề, kiểm rằng bản thô chưa bị đọc lại trong lúc dựng, rồi mới ghi bản ghi chung của cây, *sau cùng*; khi xoá thì nó đi *trước*. Nhờ vậy, thấy bản ghi chung là thấy cả cây. Chỉ sau đó job mới gửi kết quả sẵn sàng về `ingest`. Một job dựng cây bị chạy lại khi cây của đúng bản thô hiện tại đã ghi xong thì chỉ gửi lại kết quả ấy, không dựng lại.

7.3.4 Các ràng buộc và cơ chế kiểm soát

Bảng 7.10 tổng hợp các ràng buộc `document` giữ, cơ chế thực thi mỗi ràng buộc, và cách kiểm chứng. Như ở `agent`, cột cuối dùng hai trong ba loại đã giải thích trước bảng 7.2.

Bảng 7.10: Ràng buộc `document` giữ, cơ chế thực thi, và cách kiểm chứng

Ràng buộc	Cơ chế thực thi	Kiểm chứng
Phân quyền		
Không có quyền vào database	Mã nguồn không nhắc tới thư viện hay thông tin kết nối database nào	`check_contract`

tiếp trang sau

Ràng buộc	Cơ chế thực thi	Kiểm chứng
Mỗi thư viện bên ngoài chỉ được dùng thẳng ở đúng một tệp: object store, đọc PDF, content store, Jev, mô hình ngôn ngữ	`storage`, `pdf`, `content_store`, `index/jev`, `index/llm`	`check_contract`
Chỉ `document` ghi content store; `backend` chỉ đọc	`agent` và `ingest` không dùng thư viện của content store; tệp duy nhất của `backend` dùng nó không gọi hàm ghi nào	`check_contract`
Tính nhất quán		
Kết quả sẵn sàng chỉ được gửi sau khi bản thô và cây đã ghi	Thứ tự trong hai worker	kiểm thử tự động
Bản ghi chung của cây ghi sau cùng, xoá đầu tiên	`content_store`	kiểm thử tự động
Tài liệu bị đọc lại (chỉ khi job dò tệp chạy lại) thì cây cũ bị xoá	`content_store`	kiểm thử tự động
Tệp hay trang không đọc nổi, cây không dựng được, content store hay `redis` hỏng tới lần thử cuối: mỗi ca thành một kết quả kèm lý do	Ba hàm nhận job, trong `ocr_handlers` và `index_handlers`	kiểm thử tự động
Điều khiển đồng thời		
Worker đọc tệp chạy một job một lúc	Cấu hình worker	kiểm thử tự động
Một cuốn chỉ có một chuỗi lô và một job dựng cây	Mã cố định của job	kiểm thử tự động
Thời gian		
Job tự dừng trước khi chạm trần thời gian của worker	Lô OCR không mở trang mới sau 90 giây, dưới trần 120 giây; job dựng cây tự cắt ở 540 giây, dưới trần 600 giây	kiểm thử tự động

Ràng buộc thời gian được kiểm chứng hẹp hơn lời của nó, ở hai chỗ. Khác phép kiểm của `agent`, không gì so mốc tự dừng của một job với trần thời gian của worker chạy nó: hạ trần trong cấu hình worker xuống dưới mốc ấy cũng không làm kiểm thử nào đỏ. Và lô OCR chỉ xem đồng hồ sau khi đọc xong một trang, nên một trang đọc lâu hơn ba mươi giây vẫn đẩy được cả lô quá trần. Ngoài bảng, việc `document` không nhận lời gọi nào từ bên ngoài đứng được nhờ hình dạng của nó: hai worker chỉ lấy job từ queue. Điều ấy chỉ được ghi trong luật của service; không gì chặn một lượt sửa sau thêm vào đó một máy chủ HTTP.

7.3.5 Xử lý lỗi

Các worker chạy trên thư viện arq, và arq chỉ chạy lại một job khi job ấy chủ động xin; một job chạm trần thời gian thì bị giết mà không ai báo gì. Vì vậy mọi cách hỏng của `document` phải tự biến thành một kết quả (bảng 7.11). Cột cuối nói giáo viên thấy gì: khi có lỗi, dòng trạng thái của tài liệu in "Xử lí lỗi. Hãy tải lại", và lý do hiện khi trỏ chuột vào nó. Một tệp hỏng hay không có chữ thì không nằm trong bảng: đó là một kết quả bình thường của việc dò tệp.

Bảng 7.11: Khi một thao tác bên trong `document` hỏng, và giáo viên thấy gì

Khi	`document` làm gì	Giáo viên thấy
Content store, object store và `redis`		
Content store hay `redis` trục trặc	Xin chạy lại: tối đa năm lần chạy kể cả lần đầu, bốn khoảng nghỉ cộng lại 50 giây; lần thứ năm vẫn hỏng thì gửi kết quả hỏng	Đang xử lý, rồi lỗi: "Kho lưu trữ đang trục trặc nên chưa xử lý xong tệp. Thử tải lại sau."
Không đọc được tệp từ object store	Lúc dò tệp: gửi kết quả hỏng ngay, không chạy lại. Giữa chuỗi lô: xin chạy lại như hàng trên	Lỗi: "Chưa đọc được tệp đã cất. Thử lại sau."
Đọc tệp		
Một trang không đọc nổi giữa một lô	Gửi kết quả hỏng, nói rõ trang nào	Lỗi: "Không đọc được trang n của tệp PDF này."
Một lô chạy quá lâu	Không mở trang mới sau 90 giây, rồi giao phần còn lại cho lô kế; một trang chậm vẫn có thể đẩy lô quá trần 120 giây, và khi ấy job bị giết	Thường không thấy gì; job bị giết thì như hàng cuối
Dựng cây		
Jev hay một mô hình hỏng, hay chưa được cấu hình	Hàm ấy đi đường lùi của nó (bảng 7.9)	Không thấy gì; cây kém hơn
Job dựng cây chạy quá lâu	Tự cắt ở 540 giây, dưới trần 600 giây, rồi gửi kết quả hỏng kèm lý do	Lỗi, như hàng dưới
Một lỗi bất ngờ khác khi dựng cây	Gửi kết quả hỏng	Lỗi: "Đã đọc được chữ nhưng chưa dựng được mục lục. Thử tải lại tệp."
Tài liệu bị đọc lại giữa lúc dựng cây	Lần đọc mới đã xong thì xin chạy lại trên bản mới; còn đang đọc thì dừng, không gửi gì, và lần đọc mới, khi xong, sẽ giao lại job dựng cây	Thường không thấy gì; job mới bị từ chối vì job cũ còn chạy thì như hàng cuối

tiếp trang sau

Khi	`document` làm gì	Giáo viên thấy
Không có bản thô nào để dựng	Dừng, không gửi gì	Như hàng cuối
Khi `document` không báo được gì		
Không báo gì quá 300 giây: job bị giết, hay `redis` chết ở lần thử cuối	`backend` đọc tài liệu ra là hỏng	Lỗi: "Xử lý tài liệu này đã dừng giữa đường. Thử tải lại."

7.3.6 Các giới hạn hiện tại

Bốn yêu cầu dưới đây đã được đặt ra cho hệ thống, nhưng `document` mới thực thi một phần. Mỗi yêu cầu có một chỗ hở cụ thể:

- **Một tài liệu đang đọc không bị báo hỏng oan** — worker đọc tệp chạy một job một lúc, nên nhiều bản chụp tải lên cùng lúc được đọc xen kẽ từng lô, và ngay một lần dò tệp có sẵn chữ cũng phải xếp sau các lô ấy. Mỗi lô mất chừng bốn mươi lăm giây, tới chín mươi giây nếu trang dày chữ, nên hai lần báo tiến độ của một cuốn cách nhau chừng ấy nhân với số cuốn đang đọc. Từ chừng bảy cuốn, hay bốn cuốn nếu trang dày chữ, khoảng cách ấy vượt ba trăm giây và `backend` đọc cuốn ấy ra là hỏng. Lỗi ấy tự hết khi lô kế của cuốn báo tiến độ, nhưng trong lúc đó giáo viên đọc "Hãy tải lại".
- **Cây mục lục đúng với cuốn sách** — ở ngưỡng hiện tại, trên cuốn duy nhất có mục lục chuẩn để so, Jev chưa làm đổi nút nào của cây, nên cây của cuốn ấy chính là cây của luật. Cả luật lẫn ngưỡng đều được chỉnh trên chính cuốn ấy, và luật còn mang mẫu riêng của một bộ sách giáo khoa, nên phép so Jev với luật chưa công bằng. Một lỗi đã gặp: trên cuốn Toán 10 tập 1, nút "Chương IV. VECTƠ" trỏ vào các trang của chương I, nên một đề "phần vectơ" ra tám câu về mệnh đề và tập hợp.
- **Đọc trọn chữ của bản chụp** — công thức chỉ có chữ OCR, và khối bảng không nhận chữ nào. Hướng đã chọn là đọc công thức lúc ra đề, chỉ ở phần tài liệu được dùng, bằng một mô hình đọc ảnh; đường ấy chưa được viết.
- **Giáo viên biết khi cây kém hơn** — bản ghi chung của cây ghi cây được xếp bằng Jev hay chỉ bằng luật, mô hình tóm tắt và mô hình đọc tên chương có được cấu hình không, và bao nhiêu dòng Jev không đủ chắc. Nhưng không đoạn mã nào của `backend` hay `agent` đọc những trường ấy, nên giáo viên không được báo; còn một mô hình hỏng lúc chạy thì bản ghi chung không ghi lại.

Tài liệu tham khảo

Báo cáo này mô tả một hệ thống đang chạy được, nên nguồn của nó là chính mã nguồn và các bản ghi quyết định đi kèm, chứ không phải tài liệu bên ngoài. Mọi con số và mọi luật nêu trong báo cáo đều có một chỗ tương ứng trong kho dưới đây.

Mã nguồn và tài liệu kỹ thuật của dự án
`https://github.com/biabeogo147/AI-Assessment-and-Feedback-Assistant`

Kho chứa năm **service** (thành phần) đã mô tả ở chương 6, mà ba trong số đó được đi vào bên trong ở chương 7, cùng ba nhóm tài liệu mà báo cáo này rút gọn từ đó:

- `docs/decisions/` — các bản ghi quyết định nghiệp vụ. Mỗi quyết định ở đó ghi cả phần *đã cân nhắc những gì* và *nơi nào thi hành nó*, hai phần mà báo cáo này không đưa vào.
- `docs/overview/` — mô tả nghiệp vụ, kiến trúc và mô hình dữ liệu. Nguồn của chương 4, 5, 6 và 7.
- `docs/diagrams/` — tệp gốc của mọi sơ đồ trong báo cáo. Các hình ở đây được *xuất ra* từ chúng chứ không vẽ lại, và một phép kiểm tự động trong kho sẽ báo đỏ nếu một sơ đồ đổi mà hình chưa xuất lại.

Các luật được canh bằng máy
`tools/check_contract.py` và `AGENTS.md` trong kho trên.